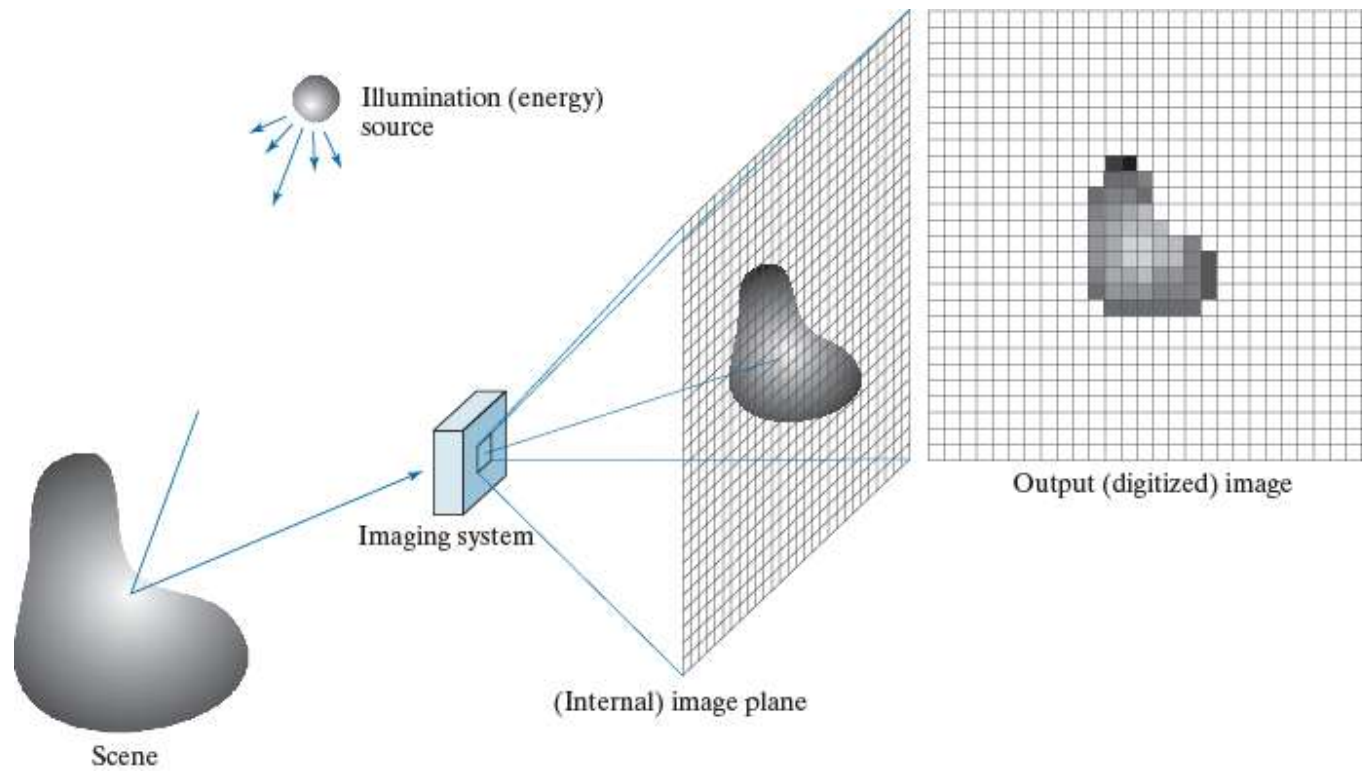


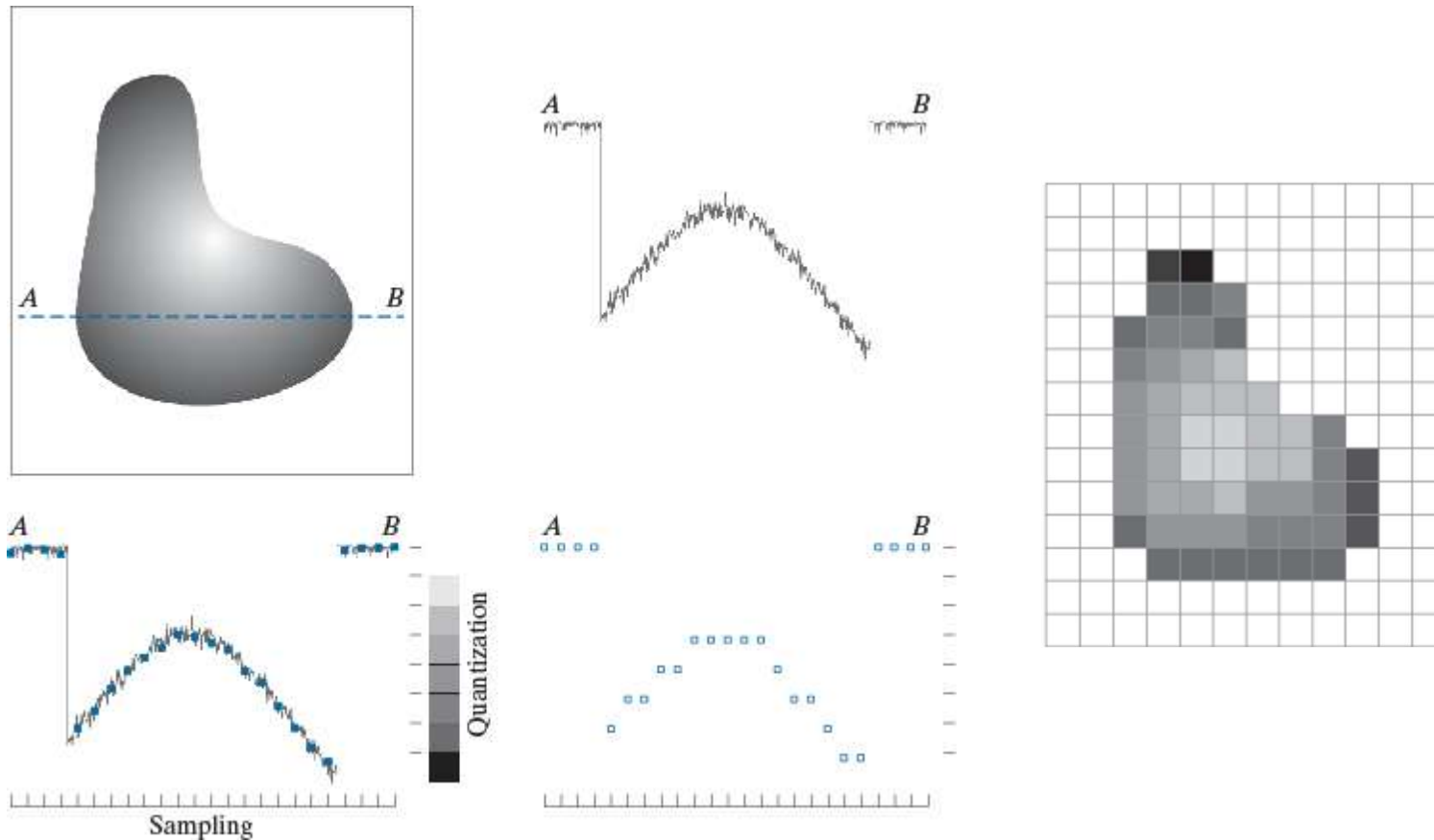
Image Acquisition, Geometric Transformations, and Image Interpolation

Digital Image Processing
ECE 253

Image acquisition



Digitization, one row of image



Number of quantization levels

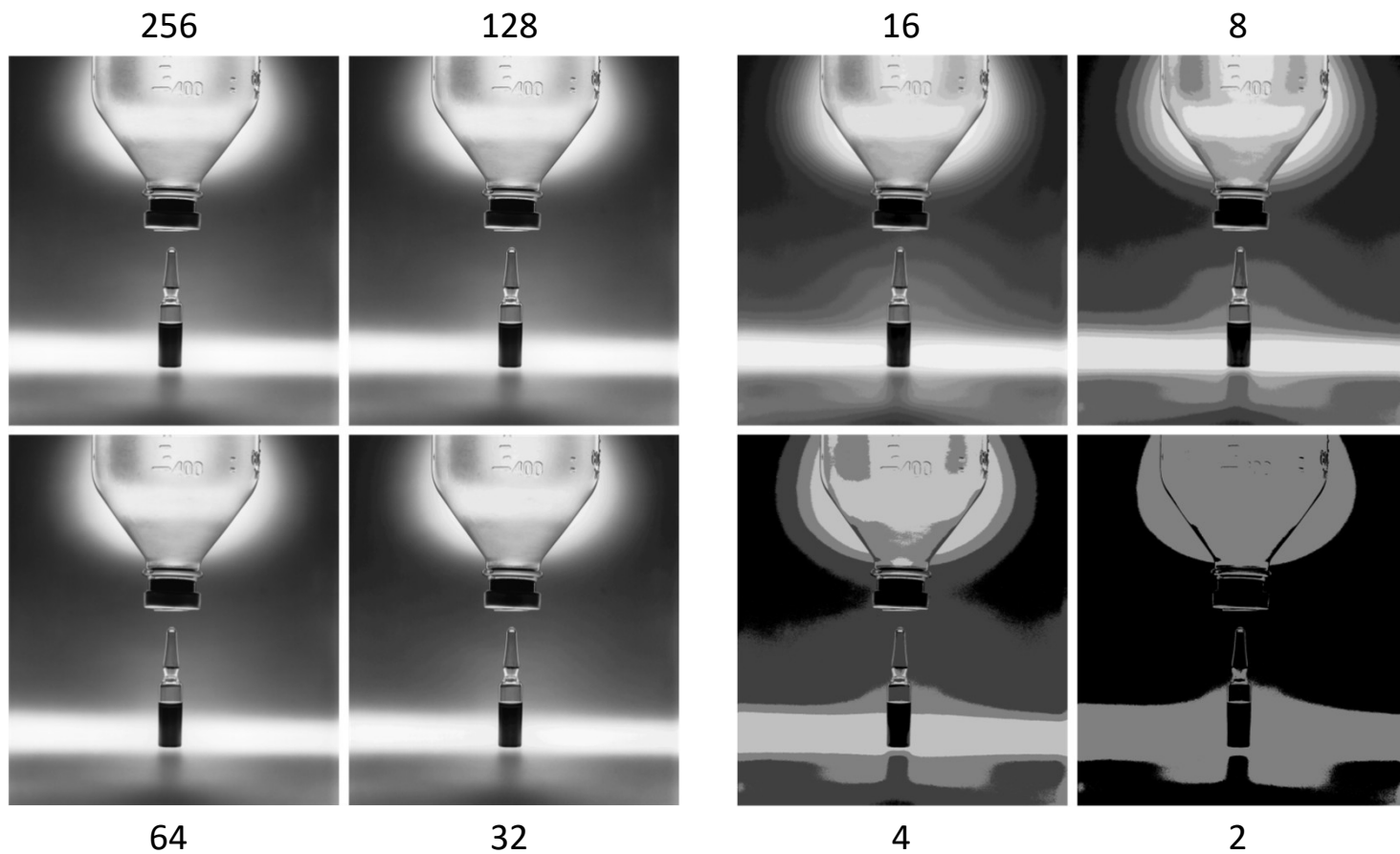
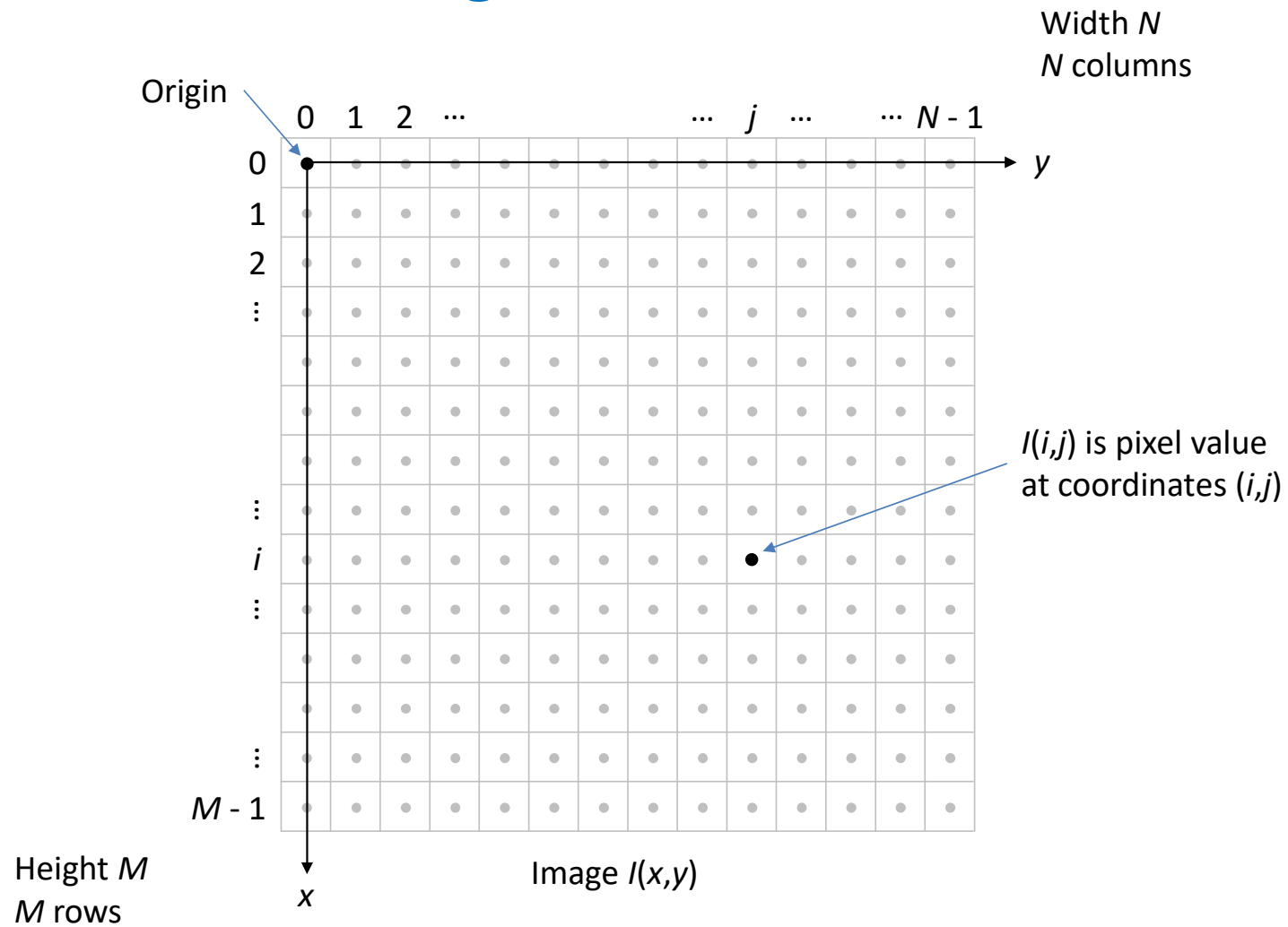
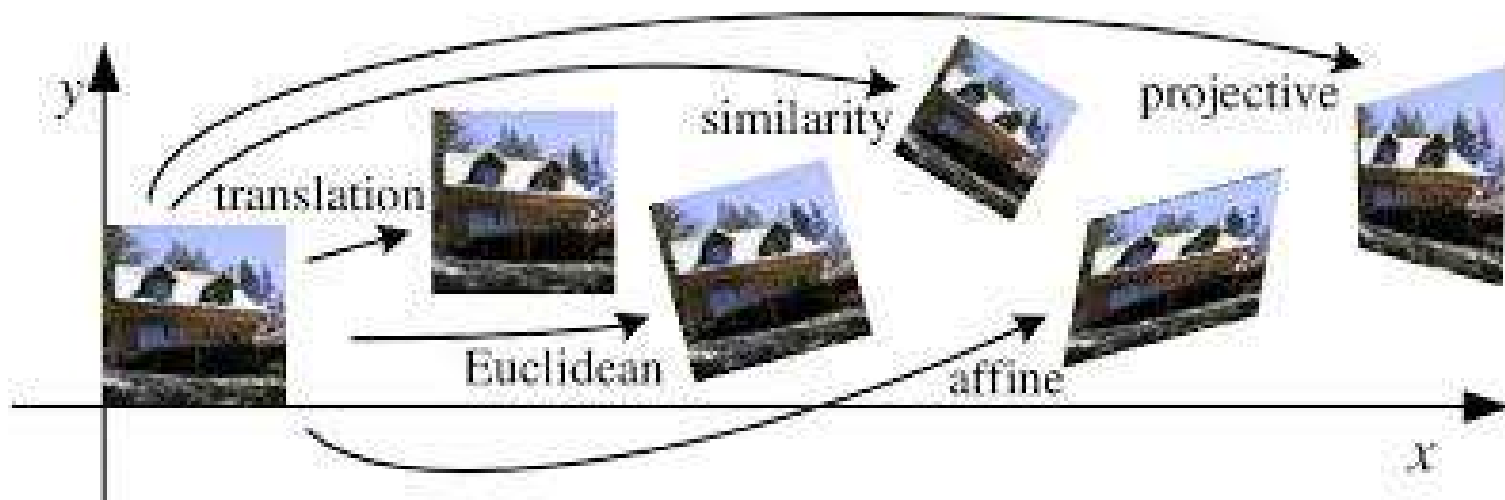


Image coordinates

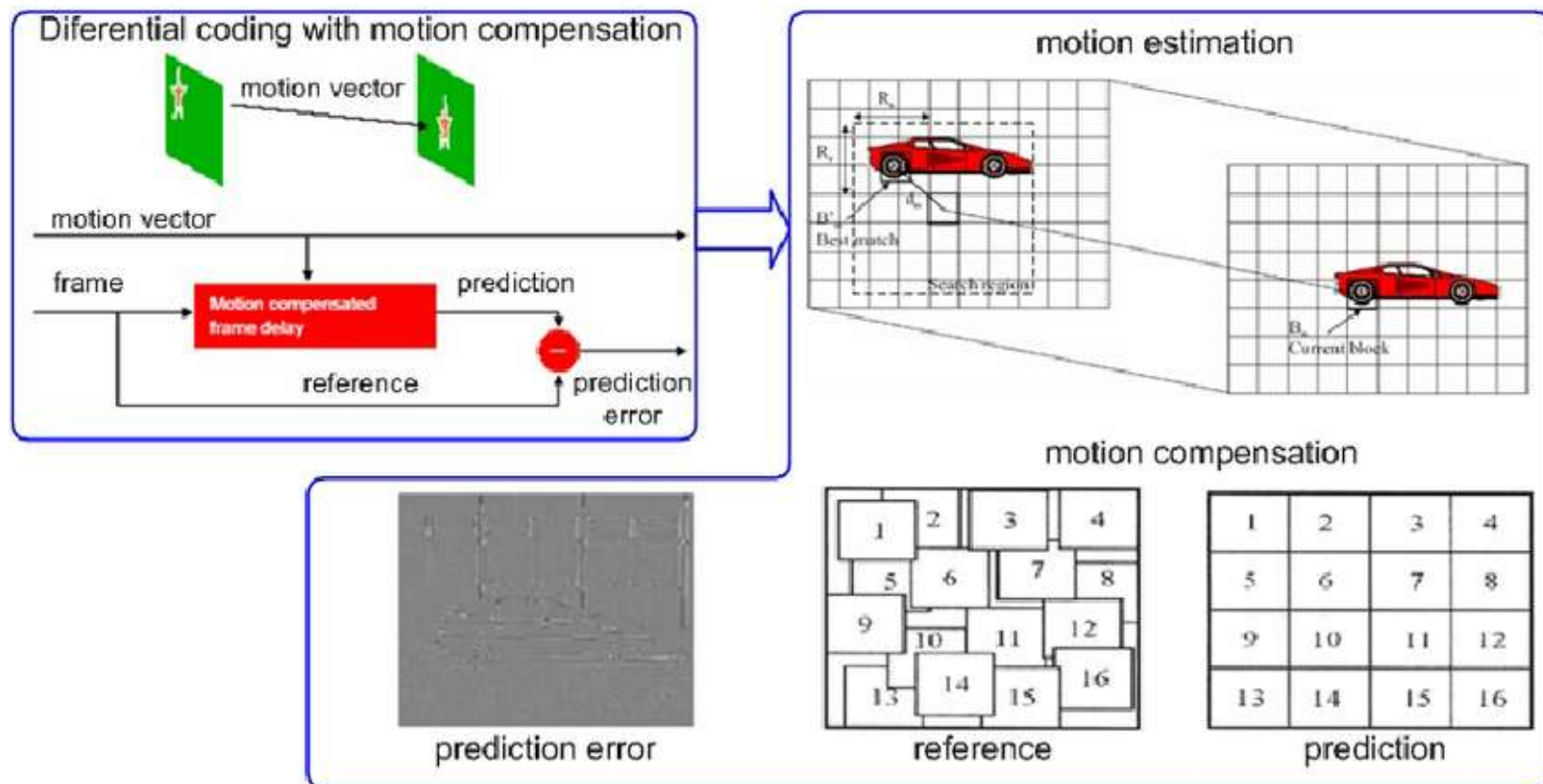


Geometric transformations



Application of Geometric Transformation

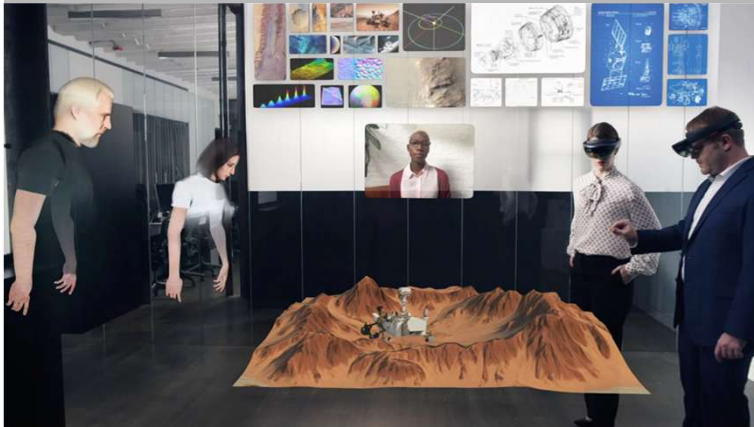
Motion-compensated Video Coding



Application of Geometric Transformation

3D Dynamic Mesh Compression

3D Virtual Meeting



Virtual Reality

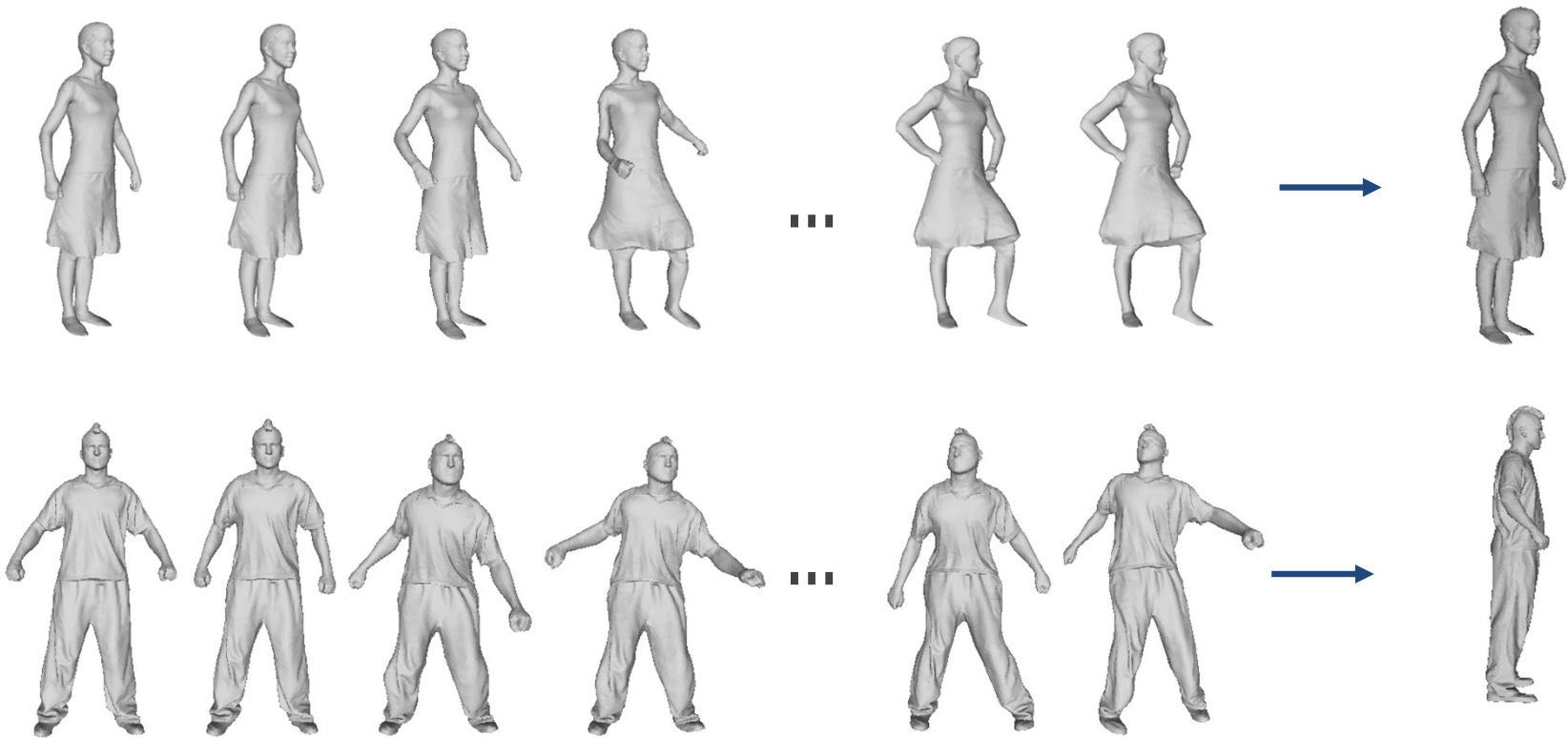


3D Gaming



3D Dynamic Mesh Examples

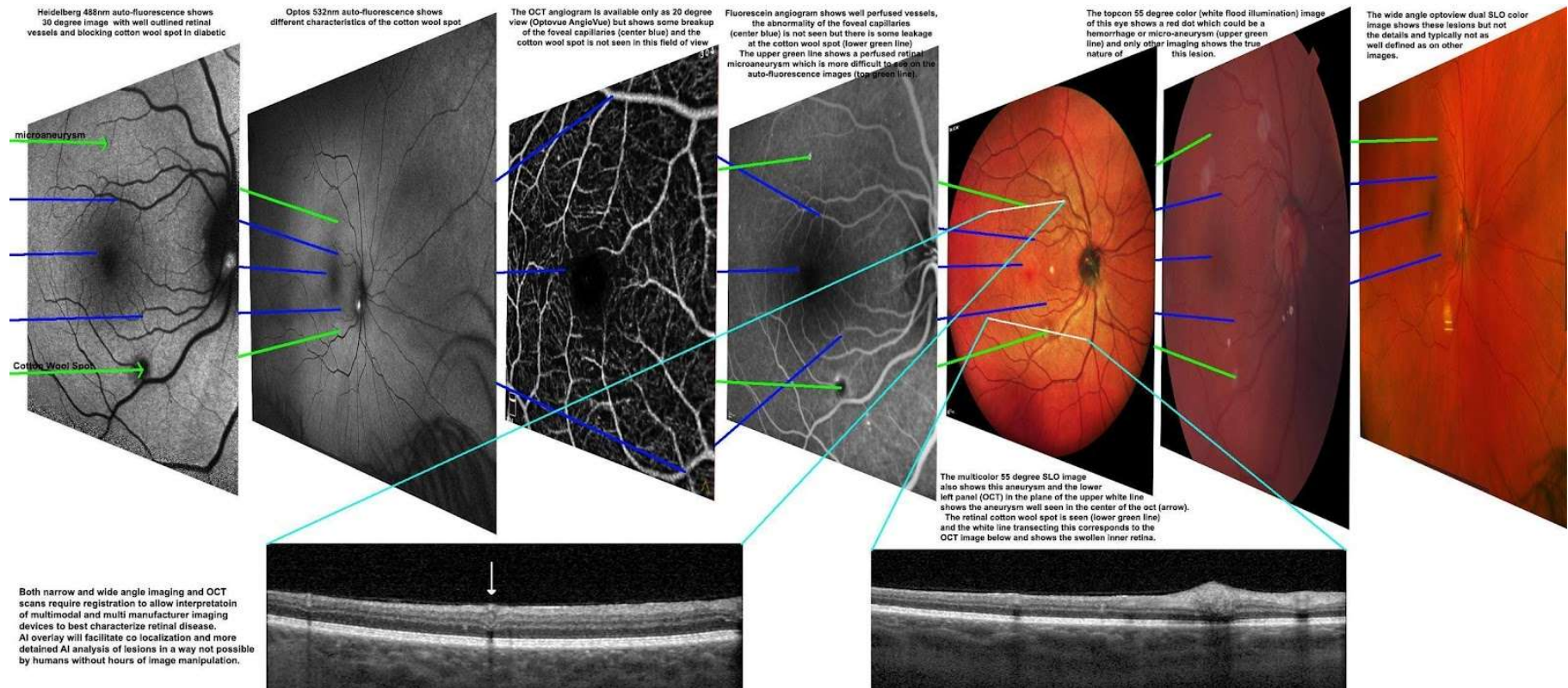
(Skipped some frames in between to show motions)



Courtesy of Huong Hoang

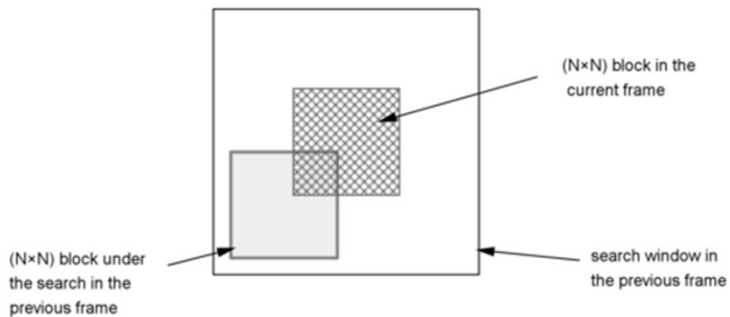
Application of Geometric Transformation

Multimodal Retina Image Registration

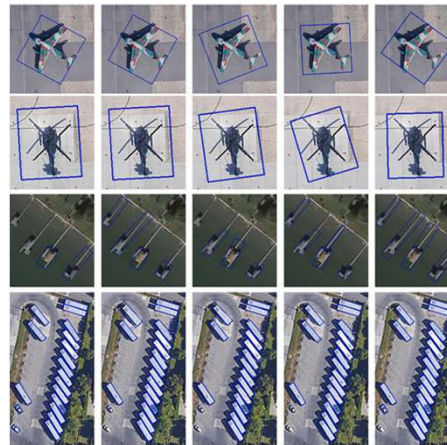


Overlay of multimodal imaging onto an SLO image. This is done manually by an ophthalmologist, doing so with OCT and the multimodal imaging would take 3-5 minutes per lesion, or over 30 minutes per eye for a diabetic with ten lesions in each eye.

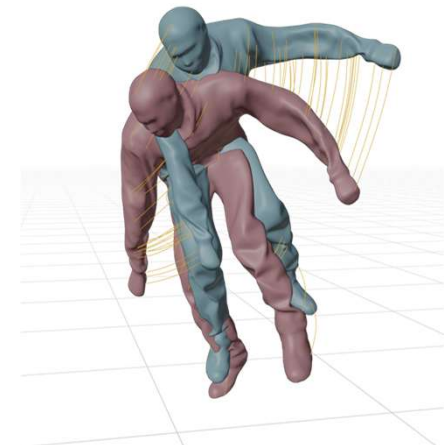
Applications of Geometric Transformations



Translations in Motion Compensation in Video Coding



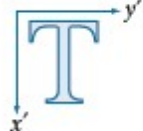
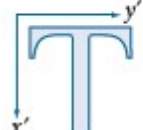
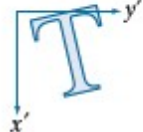
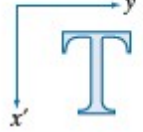
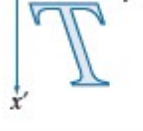
Rotations in Object Tracking



Combination of Transformations in 3D Model Deformation

Geometric transformations

TABLE 2.3
Affine

Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = y$	
Scaling/Reflection (For reflection, set one scaling factor to -1 and the other to 0)	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = c_x x$ $y' = c_y y$	
Rotation (about the origin)	$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x \cos \theta - y \sin \theta$ $y' = x \sin \theta + y \cos \theta$	
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + t_x$ $y' = y + t_y$	
Shear (vertical)	$\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + s_v y$ $y' = y$	
Shear (horizontal)	$\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = s_h x + y$	

Geometric transformations

- Translation

$$\begin{aligned}x' &= x + t_x \\ y' &= y + t_y\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{H}_t \mathbf{x}$$

- Rotation

$$\begin{aligned}x' &= x \cos \theta - y \sin \theta \\ y' &= x \sin \theta + y \cos \theta\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{H}_R \mathbf{x}$$

- Euclidean transformation

- Rotation and translation

$$\begin{aligned}x' &= x \cos \theta - y \sin \theta + t_x \\ y' &= x \sin \theta + y \cos \theta + t_y\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{H}_E \mathbf{x}$$

Geometric transformations

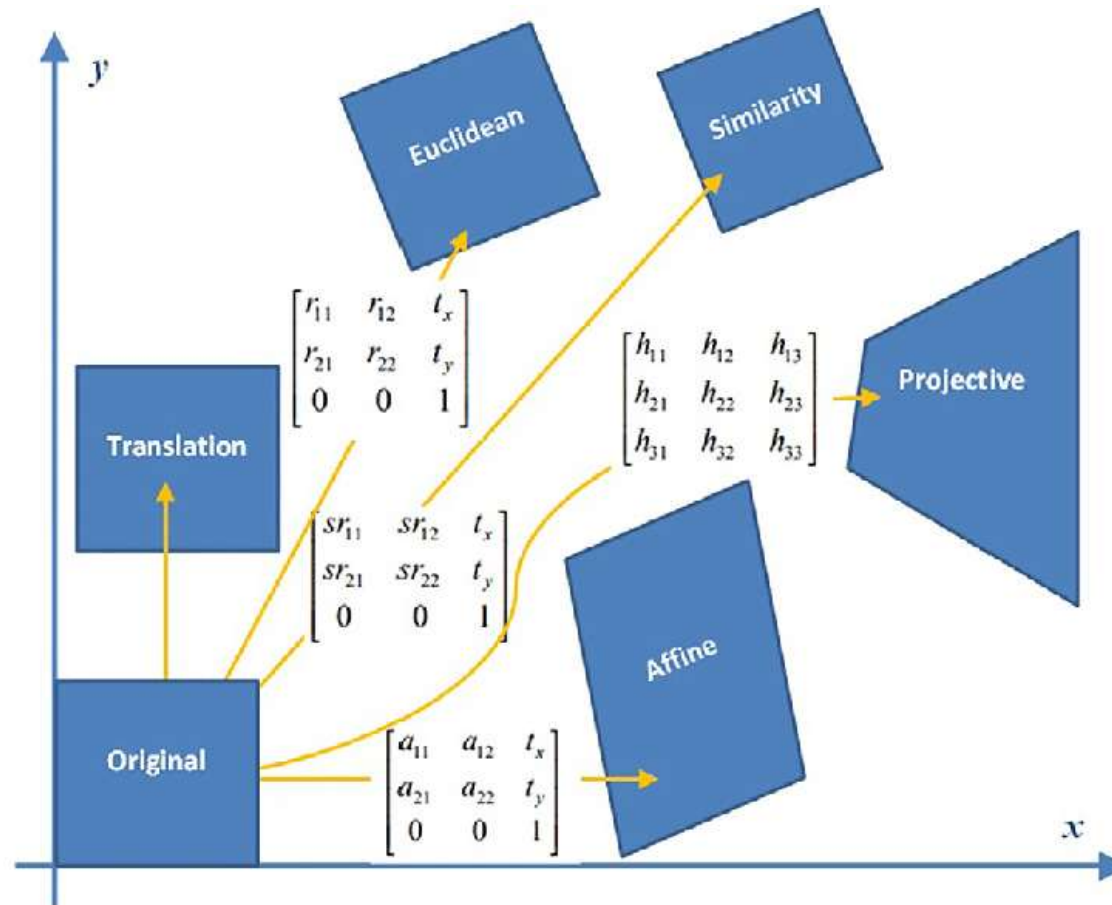
- Similarity transformation
 - Scale, rotation, and translation

$$\begin{aligned}x' &= sx \cos \theta - sy \sin \theta + t_x \\y' &= sx \sin \theta + sy \cos \theta + t_y\end{aligned}$$
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s \cos \theta & -s \sin \theta & t_x \\ s \sin \theta & s \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{H}_S \mathbf{x}$$

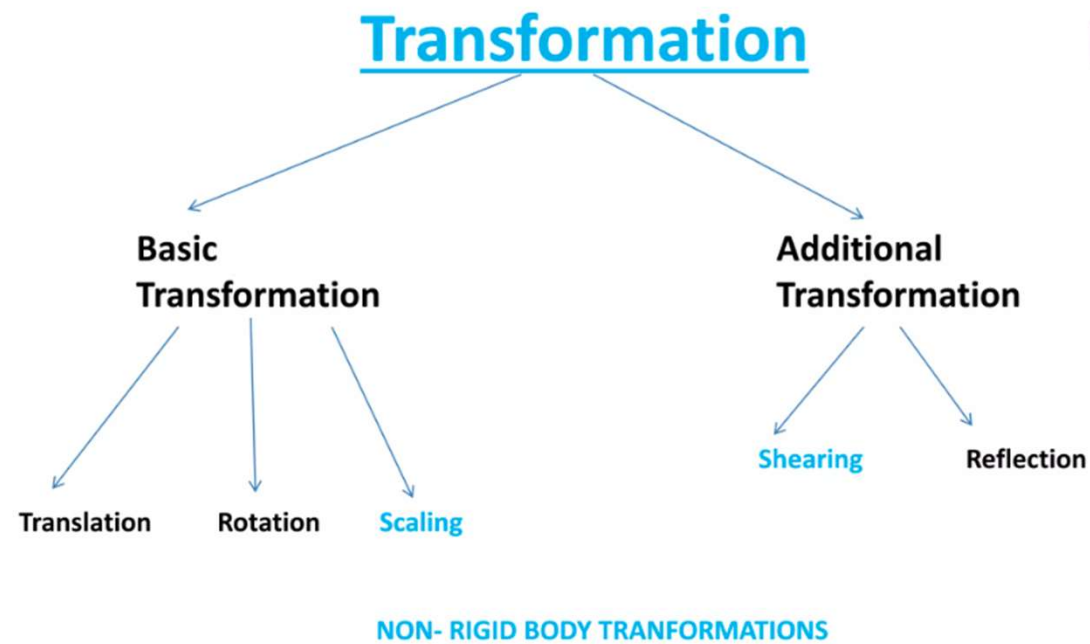
- Affine transformation
 - Linear transformation and translation

$$\begin{aligned}x' &= a_{11}x + a_{12}y + a_{13} \\y' &= a_{21}x + a_{22}y + a_{23}\end{aligned}$$
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{H}_A \mathbf{x}$$

Geometric transformations



Types of Transformations

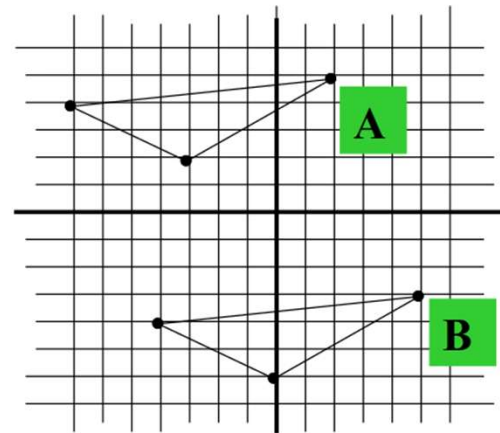


Translations

- If a figure is simply moved to another location without change to its shape or direction, it is called a translation.
- If a point is moved “a” units to the **right** and “b” units **up**, then the translated point will be at **$(x + a, y + b)$** .
- If a point is moved “a” units to the **left** and “b” units **down**, then the translated point will be at **$(x - a, y - b)$** .

Example: Image A translates to image B by moving to the right 3 units and down 8 units.

$$A(2, 5) \rightarrow B(2+3, 5-8) \rightarrow B(5, -3)$$



Courtesy of Huong Hoang

Geometric transformations - Translation

Example 1

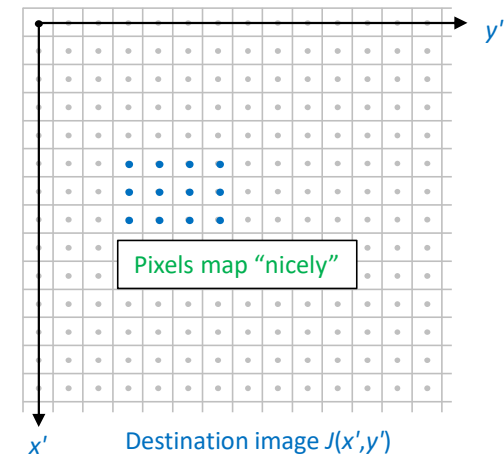
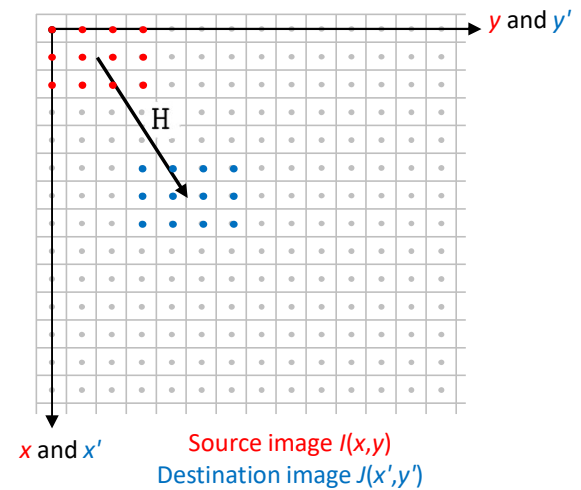
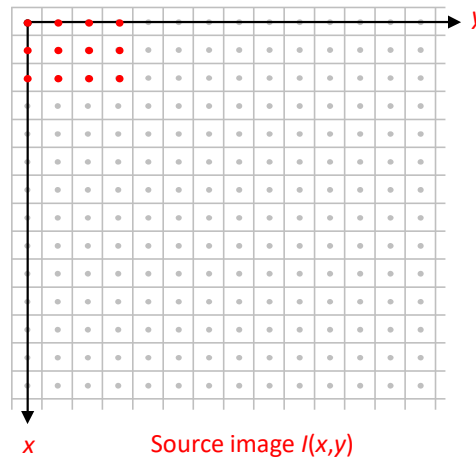
Translation

$$(t_x = 5, t_y = 3)$$

$$x' = x + t_x$$

$$y' = y + t_y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{H}_t \mathbf{x}$$



Geometric transformations - Translation

Example 2

Translation

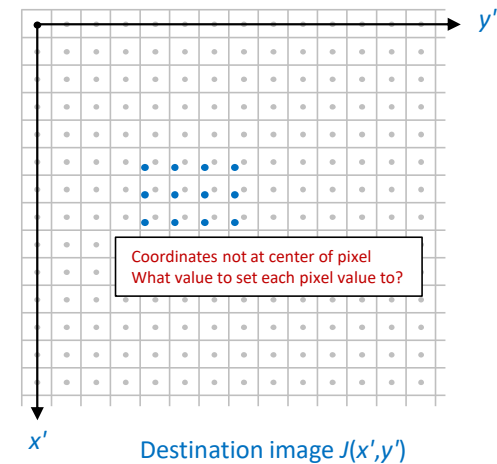
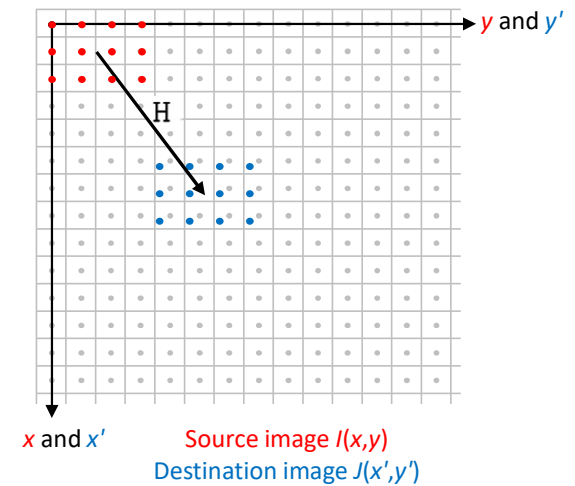
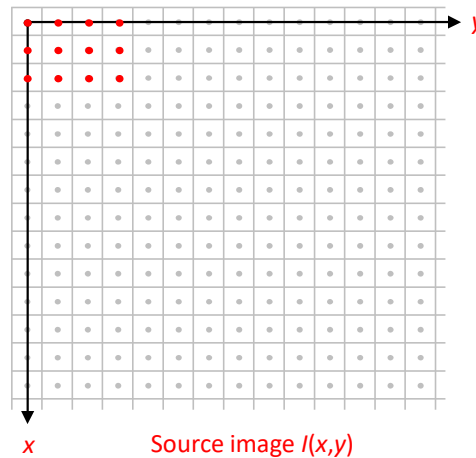
$$(t_x = 5.2, t_y = 3.6)$$

$$x' = x + t_x$$

$$y' = y + t_y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{x}' = H_t \mathbf{x}$$



Geometric transformations - Scaling

Example 3

Scaling by 2

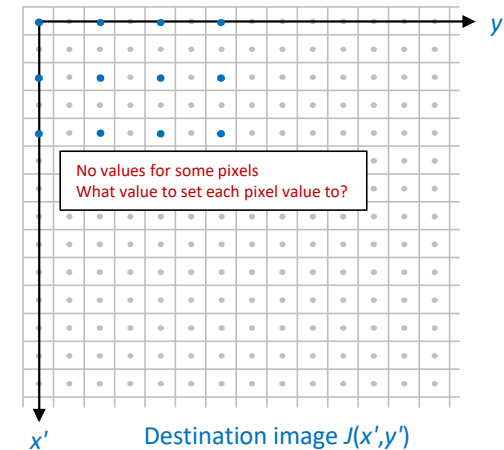
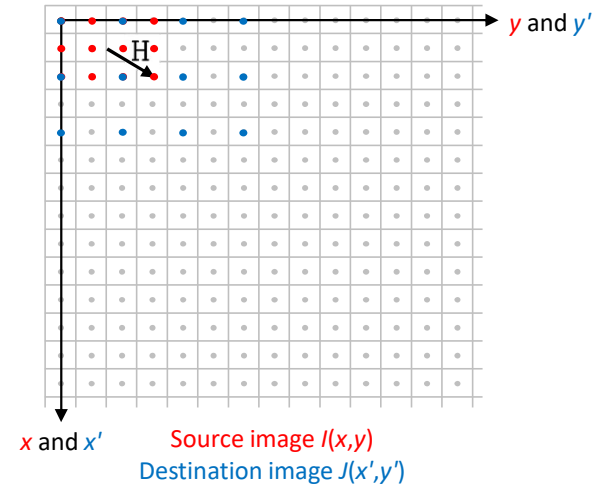
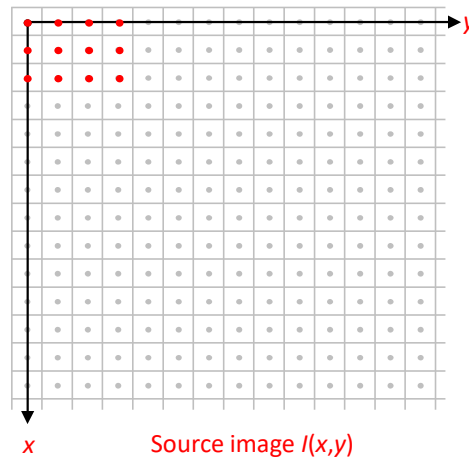
$$(c_x = 2, c_y = 2)$$

$$x' = c_x x$$

$$y' = c_y y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{x}' = H_c \mathbf{x}$$



Geometric transformations

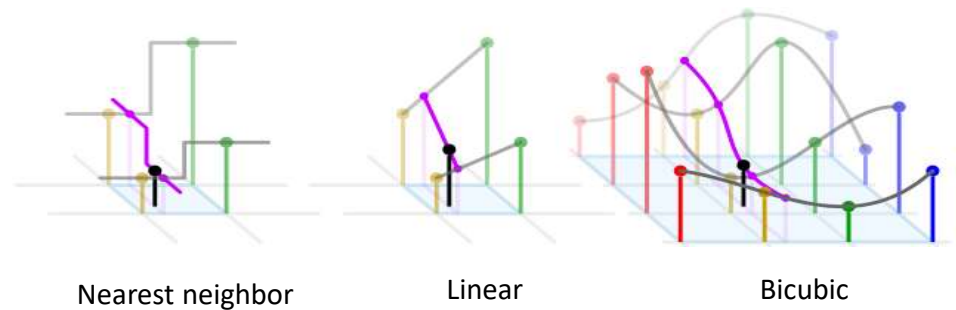
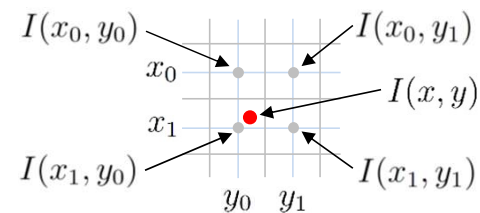
Problem

In general, pixels in the source image will not map “nicely” to pixels in the destination image

Solution

Instead of mapping each pixel in the source image to the destination image, **inverse** map each pixel coordinate in the destination image to the source image then set destination image pixel value to the source image **interpolated** pixel value at the point mapped to

Interpolation



Geometric transformations - Translation

Example 1

Translation

$$(t_x = 5, t_y = 3)$$

$$x' = x + t_x$$

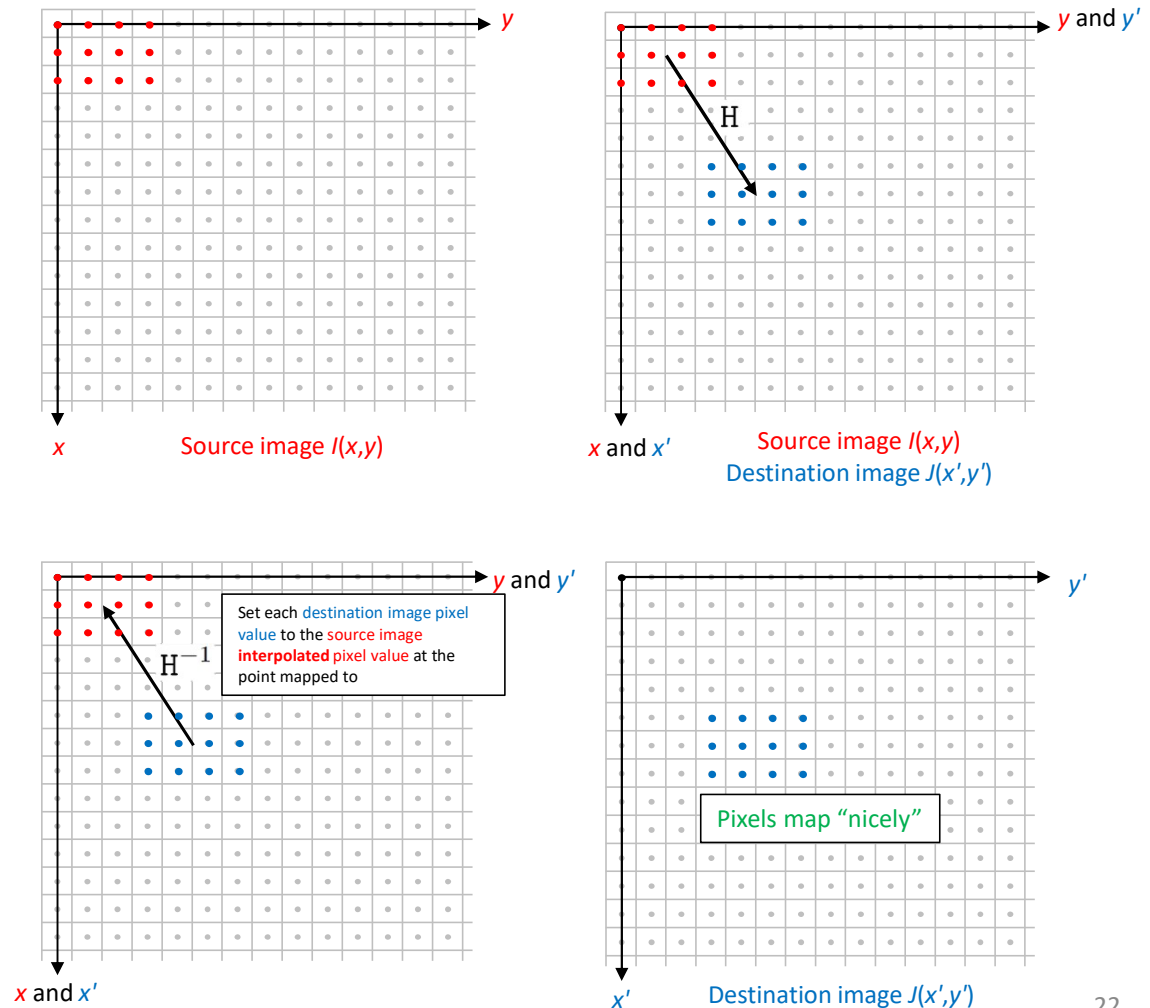
$$y' = y + t_y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{x}' = H_t \mathbf{x}$$

Backward Transform

$$\begin{aligned} \mathbf{x}' &= H \mathbf{x} \\ \mathbf{x} &= H^{-1} \mathbf{x}' \end{aligned}$$



Geometric transformations - Translation

Example 2

Translation

$$(t_x = 5.2, t_y = 3.6)$$

$$x' = x + t_x$$

$$y' = y + t_y$$

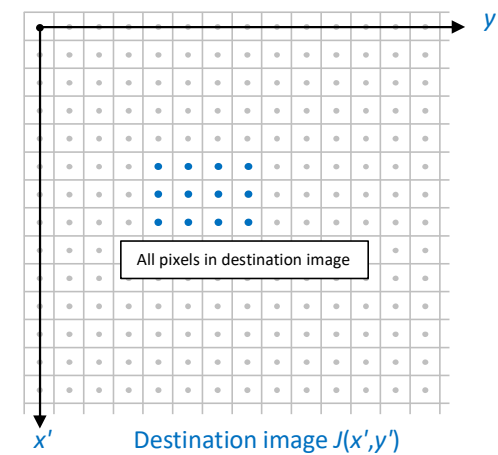
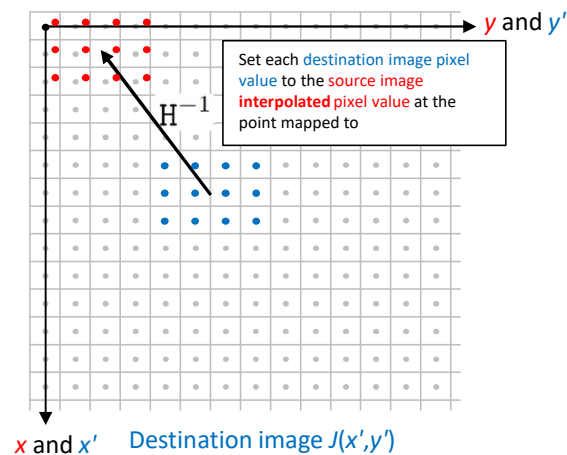
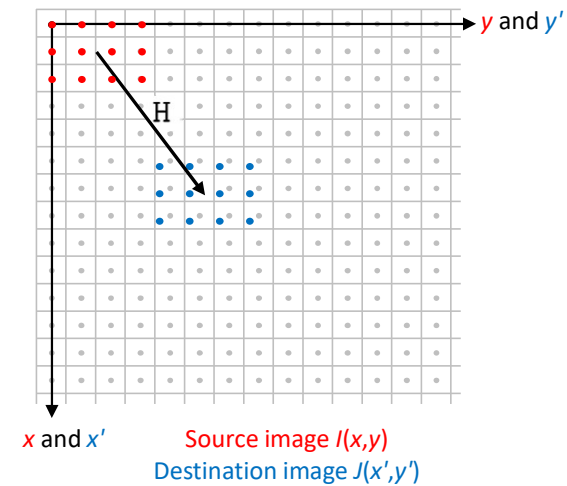
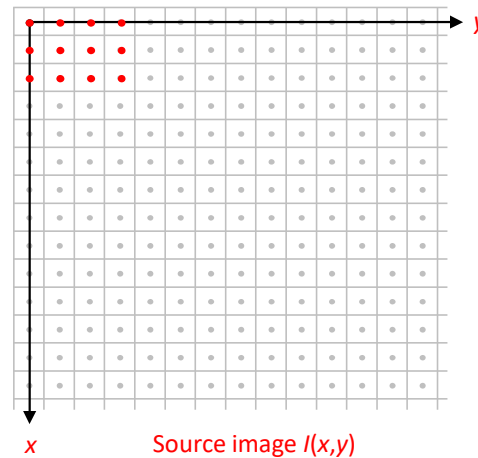
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{x}' = H_t \mathbf{x}$$

Backward Transform

$$\mathbf{x}' = H \mathbf{x}$$

$$\mathbf{x} = H^{-1} \mathbf{x}'$$



Geometric transformations - Scaling

Example 3

Scaling by 2

$$(c_x = 2, c_y = 2)$$

$$x' = c_x x$$

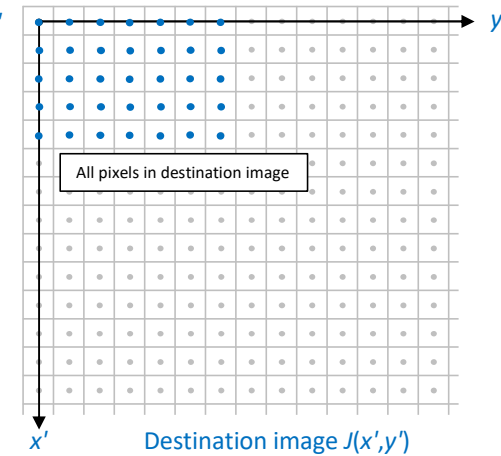
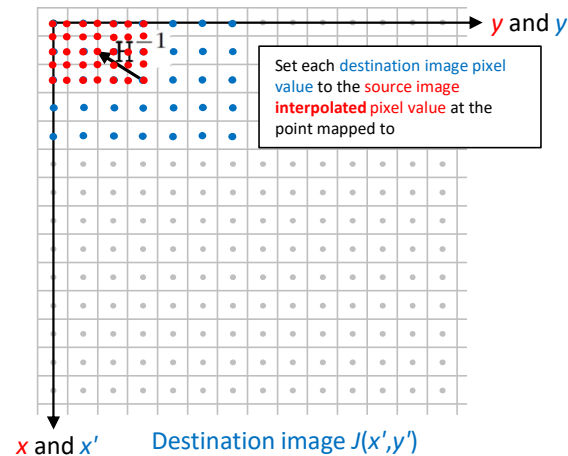
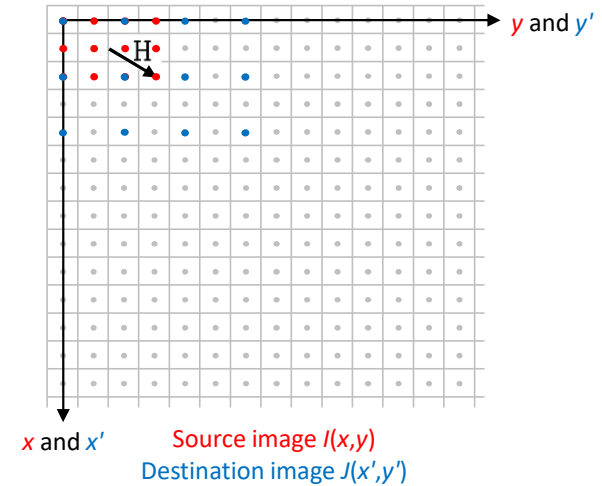
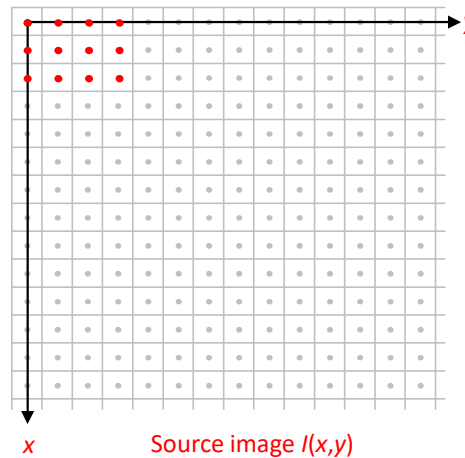
$$y' = c_y y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{x}' = \mathbf{H}_c \mathbf{x}$$

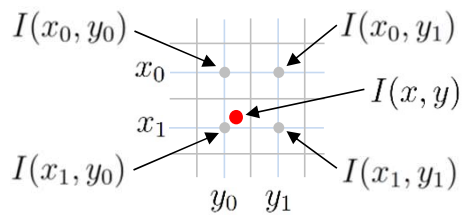
Backward Transformation

$$\begin{aligned} \mathbf{x}' &= \mathbf{H} \mathbf{x} \\ \mathbf{x} &= \mathbf{H}^{-1} \mathbf{x}' \end{aligned}$$

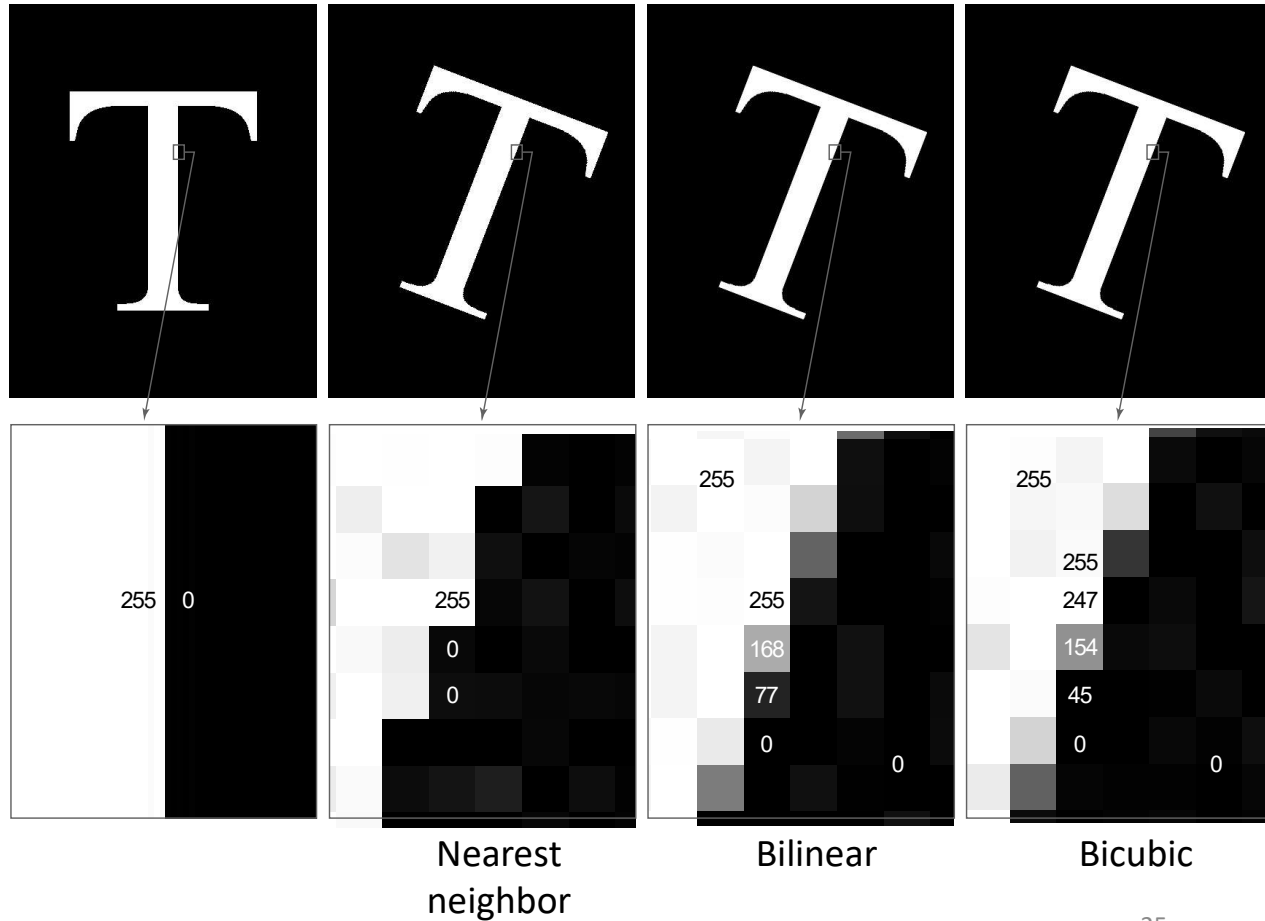
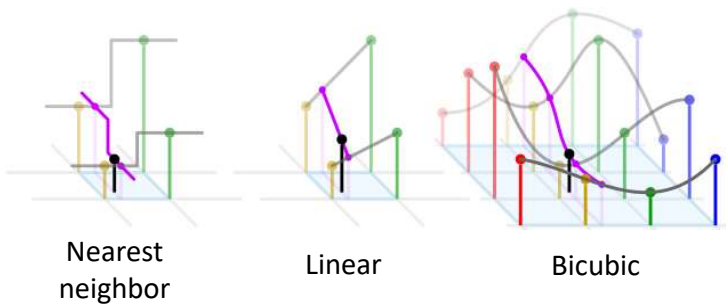


Transformation with interpolation

Interpolation



Common Methods



Transformation with interpolation

Nearest neighbor interpolation

$$H : I(x, y) \mapsto J(x', y')$$

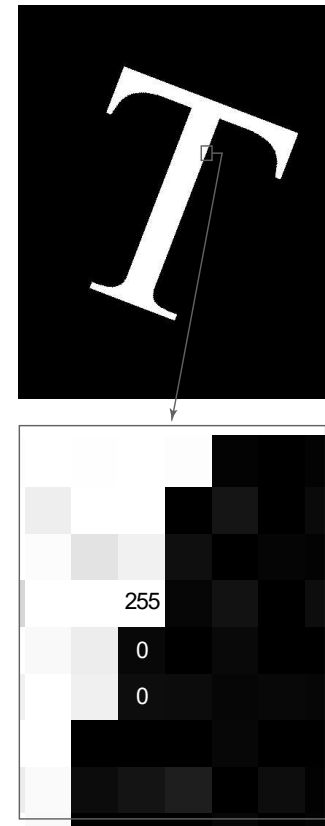
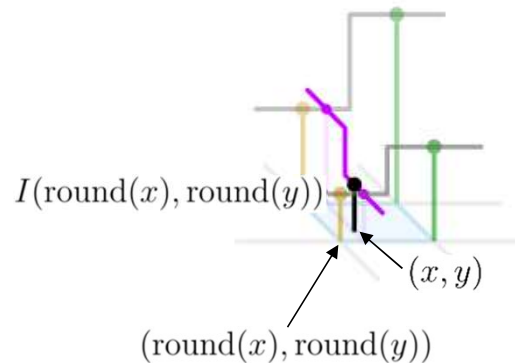
for each (x', y') in J

$$\mathbf{x} = H^{-1}\mathbf{x}'$$

$$J(x', y') = I(\text{round}(x), \text{round}(y))$$

end

Important: check that $\text{round}(x)$
and $\text{round}(y)$ are in image I



Transformation with interpolation

Linear interpolation

$$H : I(x, y) \mapsto J(x', y')$$

for each (x', y') in J

$$\mathbf{x} = H^{-1}\mathbf{x}'$$

$$x_0 = \lfloor x \rfloor$$

$$x_1 = x_0 + 1$$

$$y_0 = \lfloor y \rfloor$$

$$y_1 = y_0 + 1$$

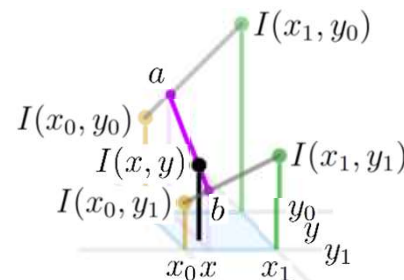
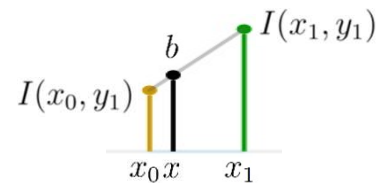
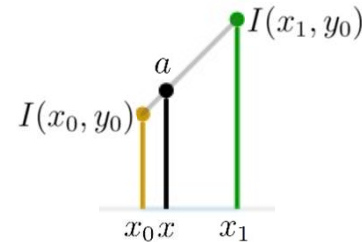
$$a = (x_1 - x)I(x_0, y_0) + (x - x_0)I(x_1, y_0)$$

$$b = (x_1 - x)I(x_0, y_1) + (x - x_0)I(x_1, y_1)$$

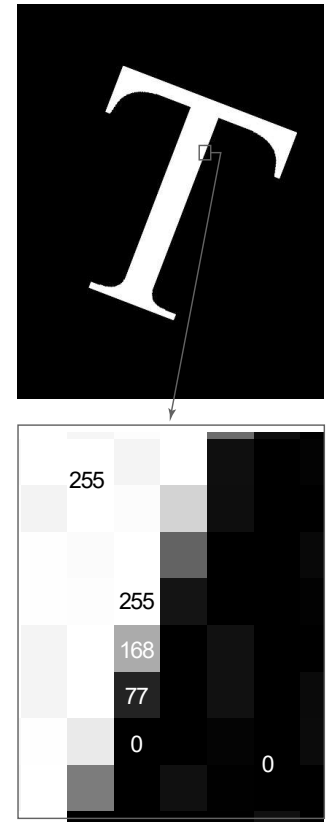
$$J(x', y') = (y_1 - y)a + (y - y_0)b$$

end

Important: check that x_0, x_1, y_0 , and y_1 are in image I



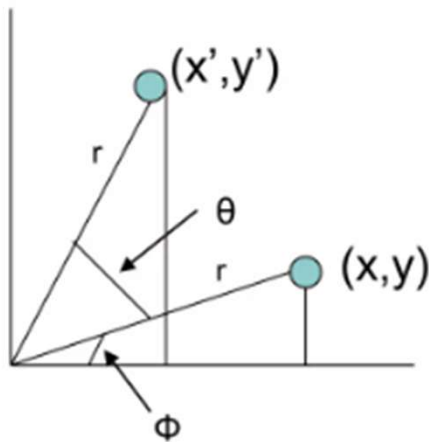
Note x and y axes are swapped



Transforming an image multiple times

- Interpolation is an estimation method
 - Estimation error
- Pixel values in transformed images include estimation error
- Transforming a transformed image (e.g., translate then rotate)
 - Estimate pixel values from estimated pixel values (i.e., compound estimation error)
- The more times an image is transformed, the greater the pixel value error

Rotations



The **original coordinates** of the point in Polar Coordinates are

$$x = r \cos (\Phi)$$

$$y = r \sin (\Phi)$$

$$x' = r \cos (\Phi + \theta) = r \cos \Phi \cos \theta - r \sin \Phi \sin \theta$$

$$y' = r \sin (\Phi + \theta) = r \cos \Phi \sin \theta + r \sin \Phi \cos \theta$$

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$P' = R.P$$

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Rotations - Examples

Rotate unit vectors **[1 0]** and **[0 1]**, by:

- 90°:

$$R(90^\circ) = \begin{bmatrix} \cos(90^\circ) & -\sin(90^\circ) \\ \sin(90^\circ) & \cos(90^\circ) \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

A vector **[x y]** is mapped to **[-y x]**.

x-axis unit vector: The vector $[1 \ 0]^T$ is mapped to:

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

y-axis unit vector: The vector $[0 \ 1]^T$ is mapped to:

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

- 180°:

$$R(180^\circ) = \begin{bmatrix} \cos(180^\circ) & -\sin(180^\circ) \\ \sin(180^\circ) & \cos(180^\circ) \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

A vector **[x y]** is mapped to **[-x -y]**.

x-axis unit vector: The vector $[1 \ 0]^T$ is mapped to:

$$\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

y-axis unit vector: The vector $[0 \ 1]^T$ is mapped to:

$$\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

Composition of transformations

Example: rotating an image about its center

– Remember, rotation is about the origin

$$\begin{aligned}x' &= x \cos \theta - y \sin \theta \\y' &= x \sin \theta + y \cos \theta\end{aligned}\qquad \begin{aligned}\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} &= \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \\ \mathbf{x}' &= \mathbf{H}_R \mathbf{x}\end{aligned}$$

1. Translation such that image center is mapped to origin
2. Rotation (about origin)
3. Translation such that origin is mapped to image center determined in step 1

Composition of transformations

Compose multiple transformations and use result to transform an image a single time

First
transformation

$$H_1 : \mathbf{x} \mapsto \mathbf{x}'$$
$$\mathbf{x}' = H_1 \mathbf{x}$$

Second
transformation

$$H_2 : \mathbf{x}' \mapsto \mathbf{x}''$$
$$\mathbf{x}'' = H_2 \mathbf{x}'$$

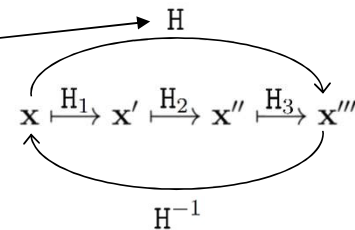
Third
transformation

$$H_3 : \mathbf{x}'' \mapsto \mathbf{x}'''$$
$$\mathbf{x}''' = H_3 \mathbf{x}''$$

$$\mathbf{x}''' = H_3 \mathbf{x}''$$
$$\mathbf{x}''' = H_3 H_2 \mathbf{x}'$$
$$\mathbf{x}''' = H_3 H_2 H_1 \mathbf{x}$$
$$\mathbf{x}''' = H \mathbf{x} \text{ where } H = H_3 H_2 H_1$$

“right to left”

Use this to
transform image

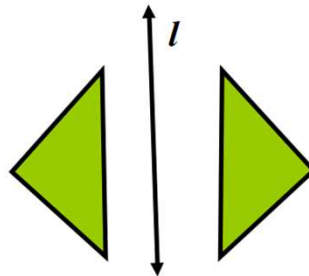


Reflections

You can reflect a figure using a line or a point. All measures (lines and angles) are preserved but in a mirror image.

Example: The figure is reflected across line l .

You could fold the picture along line l and the left figure would coincide with the corresponding parts of right figure.



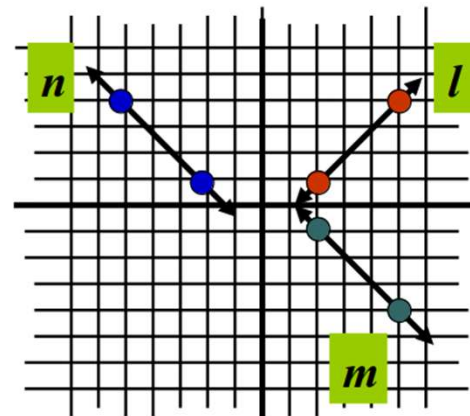
Reflections

Reflection across the x-axis: the x values stay the same and the y values change sign. $(x, y) \rightarrow (x, -y)$

Reflection across the y-axis: the y values stay the same and the x values change sign. $(x, y) \rightarrow (-x, y)$

Example: In this figure, line l :

- reflects across the y axis to line n
 $(2, 1) \rightarrow (-2, 1)$ & $(5, 4) \rightarrow (-5, 4)$
- reflects across the x axis to line m
 $(2, 1) \rightarrow (2, -1)$ & $(5, 4) \rightarrow (5, -4)$



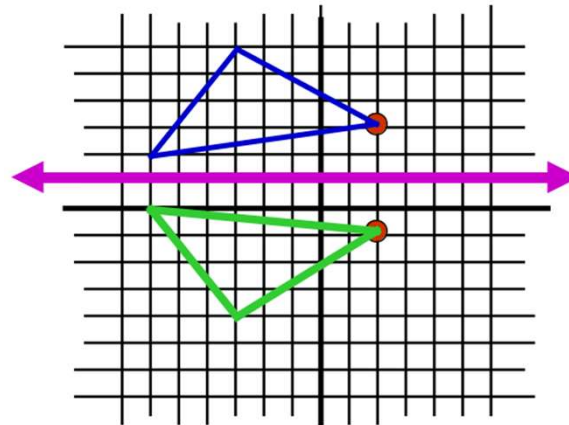
Reflections across specific lines:

To reflect a figure across the line **$y = a$ or $x = a$** , mark the corresponding points equidistant from the line.

i.e. If a point is 2 units above the line its corresponding image point must be 2 points below the line.

Example: Reflect the fig. across the line $y = 1$.

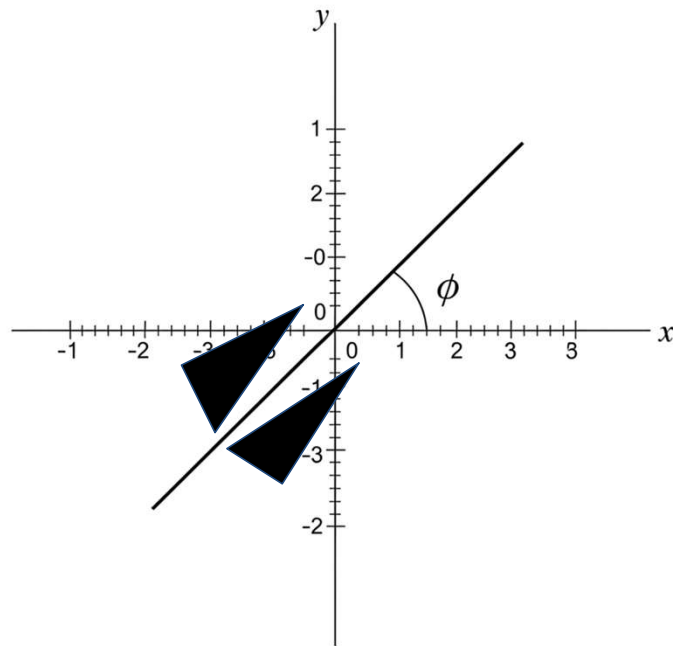
- $(2,3) \rightarrow (2,-1)$
- $(-3,6) \rightarrow (-3,-4)$
- $(-6,2) \rightarrow (-6,0)$



Reflection Matrix

The reflection matrix for a line at an angle ϕ with respect to the x-axis is:

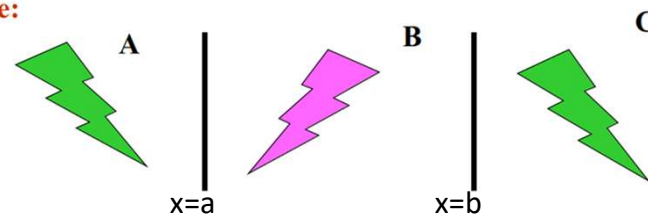
$$H(\phi) = \begin{bmatrix} \cos(2\phi) & \sin(2\phi) \\ \sin(2\phi) & -\cos(2\phi) \end{bmatrix}$$



Courtesy of Huong Hoang

Composite Reflections

Example:



If an image is reflected over a line and then that image is reflected over a parallel line (called a composite reflection), it results in a translation.

1. **First Reflection:** Reflecting a point (x, y) across the line $x = a$ moves it to a new point (x', y') .
The distance from (x, y) to the line is $x - a$, so the reflected point is at $a - (x - a) = 2a - x$.

$$(x, y) \xrightarrow{\text{Reflect over } x=a} (2a - x, y)$$

2. **Second Reflection:** Now, we reflect this new point $(2a - x, y)$ across the second line, $x = b$.

$$(2a - x, y) \xrightarrow{\text{Reflect over } x=b} (2b - (2a - x), y)$$

3. **The Result:** Simplifying the final coordinates:

$$(2b - 2a + x, y) = (x + 2(b - a), y)$$

The amount of the shift is $2(b-a)$, which is twice the distance between the two parallel lines.

Rotation as Two Reflections

The reflection matrix for a line at an angle ϕ with respect to the x-axis is given by:

- **First Reflection:** $H_1 = H(\phi_1)$.
- **Second Reflection:** $H_2 = H(\phi_2)$.

$$H(\phi) = \begin{bmatrix} \cos(2\phi) & \sin(2\phi) \\ \sin(2\phi) & -\cos(2\phi) \end{bmatrix}$$

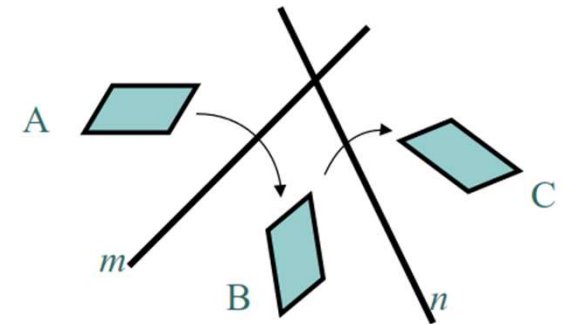


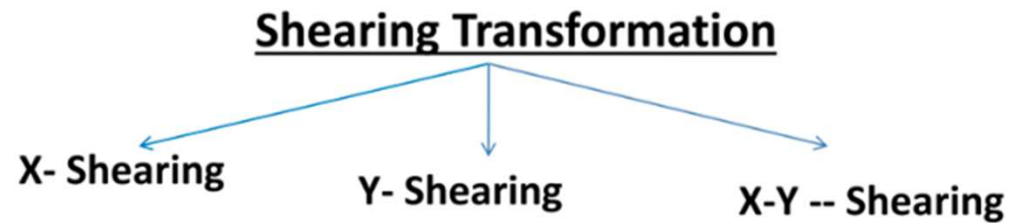
Image A **reflects** over line m to B, image B **reflects** over line n to C. Image C is **a rotation** of image A.

The Combined Transformation

$$\begin{aligned} R(\theta) = H_2 H_1 &= \begin{bmatrix} \cos(2\phi_2) & \sin(2\phi_2) \\ \sin(2\phi_2) & -\cos(2\phi_2) \end{bmatrix} \begin{bmatrix} \cos(2\phi_1) & \sin(2\phi_1) \\ \sin(2\phi_1) & -\cos(2\phi_1) \end{bmatrix} \\ &= \begin{bmatrix} \cos(2\phi_2) \cos(2\phi_1) + \sin(2\phi_2) \sin(2\phi_1) & \cos(2\phi_2) \sin(2\phi_1) - \sin(2\phi_2) \cos(2\phi_1) \\ \sin(2\phi_2) \cos(2\phi_1) - \cos(2\phi_2) \sin(2\phi_1) & \sin(2\phi_2) \sin(2\phi_1) + \cos(2\phi_2) \cos(2\phi_1) \end{bmatrix} \\ &= \begin{bmatrix} \cos(2\phi_2 - 2\phi_1) & -\sin(2\phi_2 - 2\phi_1) \\ \sin(2\phi_2 - 2\phi_1) & \cos(2\phi_2 - 2\phi_1) \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad \text{such that } \phi_2 - \phi_1 = \theta/2. \end{aligned}$$

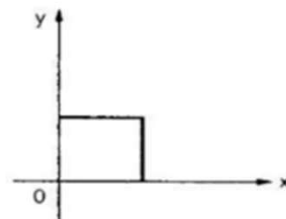
Shearing

- Shearing is also known as Skewing
- It is a transformation that slants the shape of an object

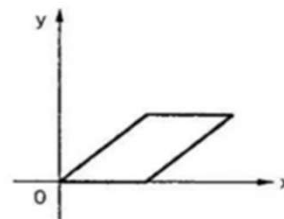


X-Shearing

- X-shear preserves the Y coordinate and changes are made to the X coordinates.



(a) Original object



(b) Object after x shear

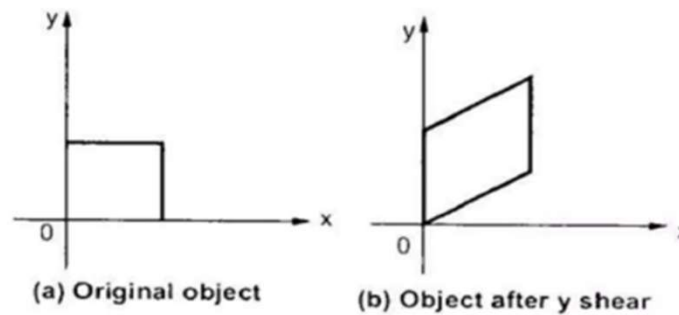
$$\bullet \quad X_{sh} = \begin{bmatrix} 1 & 0 \\ Sh_x & 1 \end{bmatrix}^T$$

$$X' = X + (Sh_x * Y)$$

$$Y' = Y$$

Y-shearing

- Y-shear preserves the X coordinate and changes are made to Y coordinates.



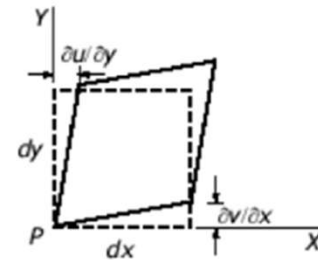
$$\bullet Y_{sh} = \begin{bmatrix} 1 & Sh_y \\ 0 & 1 \end{bmatrix}^T$$

$$Y' = Y + (Sh_y * X)$$

$$X' = X$$

X-Y-shearing

- Here, both coordinates change.



$$\bullet \quad XY_{sh} = \begin{pmatrix} 1 & Sh_y \\ Sh_x & 1 \end{pmatrix}^T$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & sh_x \\ sh_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$Y' = Y + (Sh_y * X)$$

$$X' = X + (Sh_x * Y)$$

Affine Transformation Decomposition

The full affine transformation matrix A in 2D can be written as:

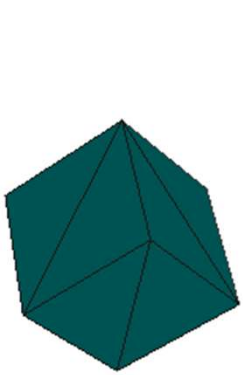
$$A = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} = T \cdot R \cdot S_{scale} \cdot S_{shear}$$

where

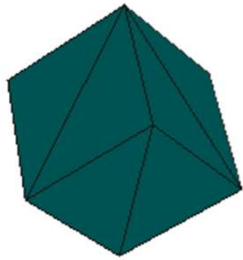
$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad S_{scale} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad S_{shear} = \begin{bmatrix} 1 & k_x & 0 \\ k_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **When Rotation is Disabled:** The rotation matrix is an identity matrix ($R=I$), by setting the rotation angles $\theta=0$.
- **When Translation is Disabled:** The translation matrix is an identity matrix ($T=I$), by setting the translation parameters $t_x=t_y=0$.
- **When Scaling is Disabled:** The scaling matrix is an identity matrix ($S_{scale}=I$), by setting the scaling parameters $s_x=s_y=1$.
- **When Shearing is Disabled:** The shearing matrix is an identity matrix ($S_{shear}=I$), by setting the shear parameters $k_x=k_y=0$.

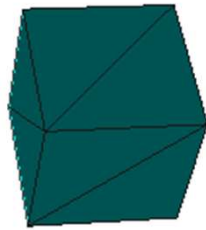
Affine Transformations - 3D Examples



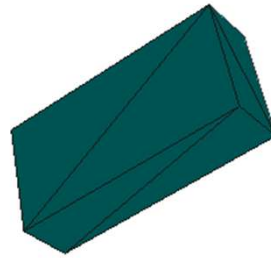
Original



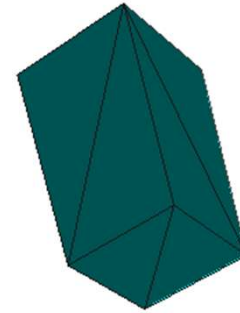
Translation
Combined



Rotation



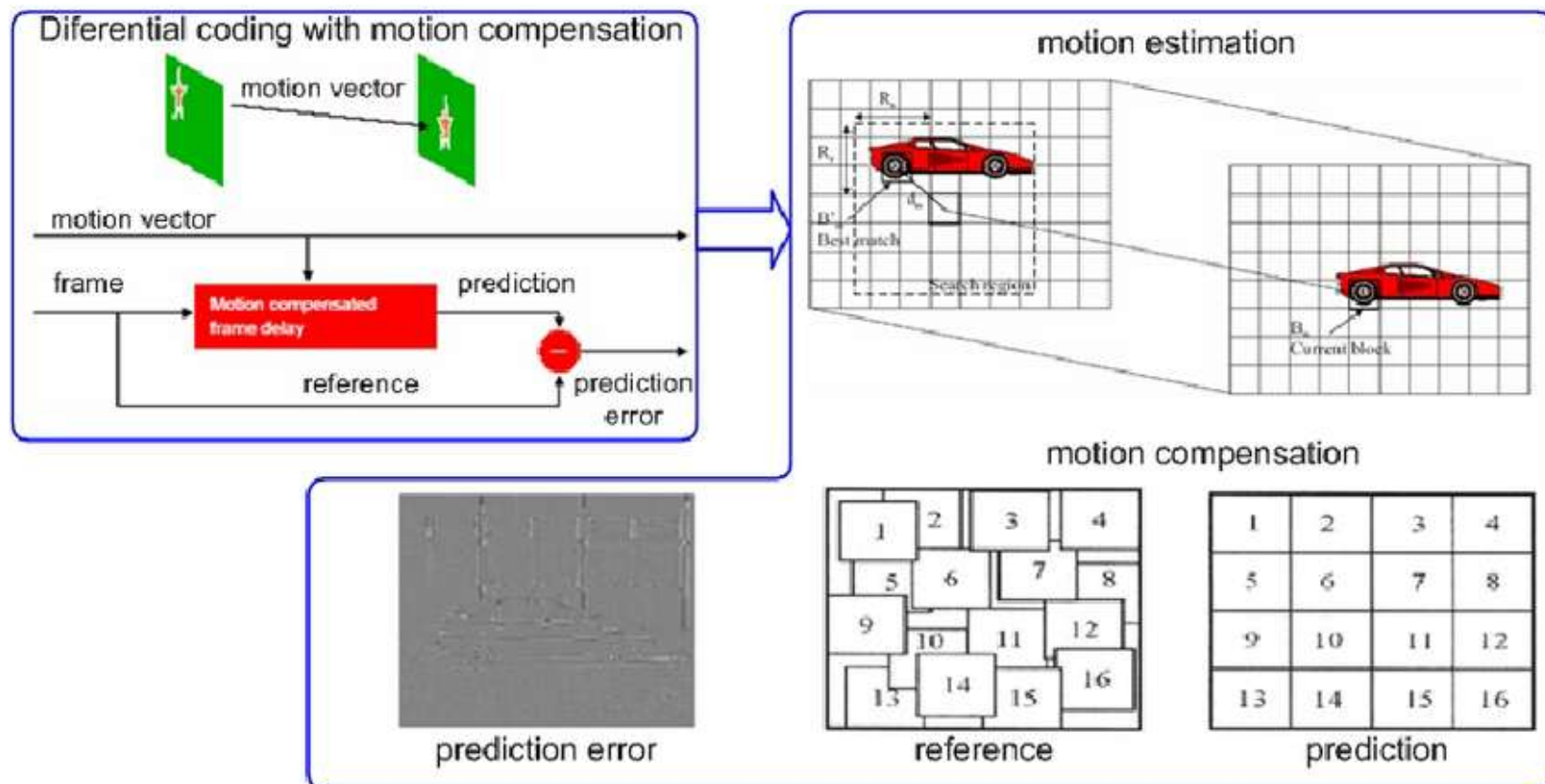
Scaling



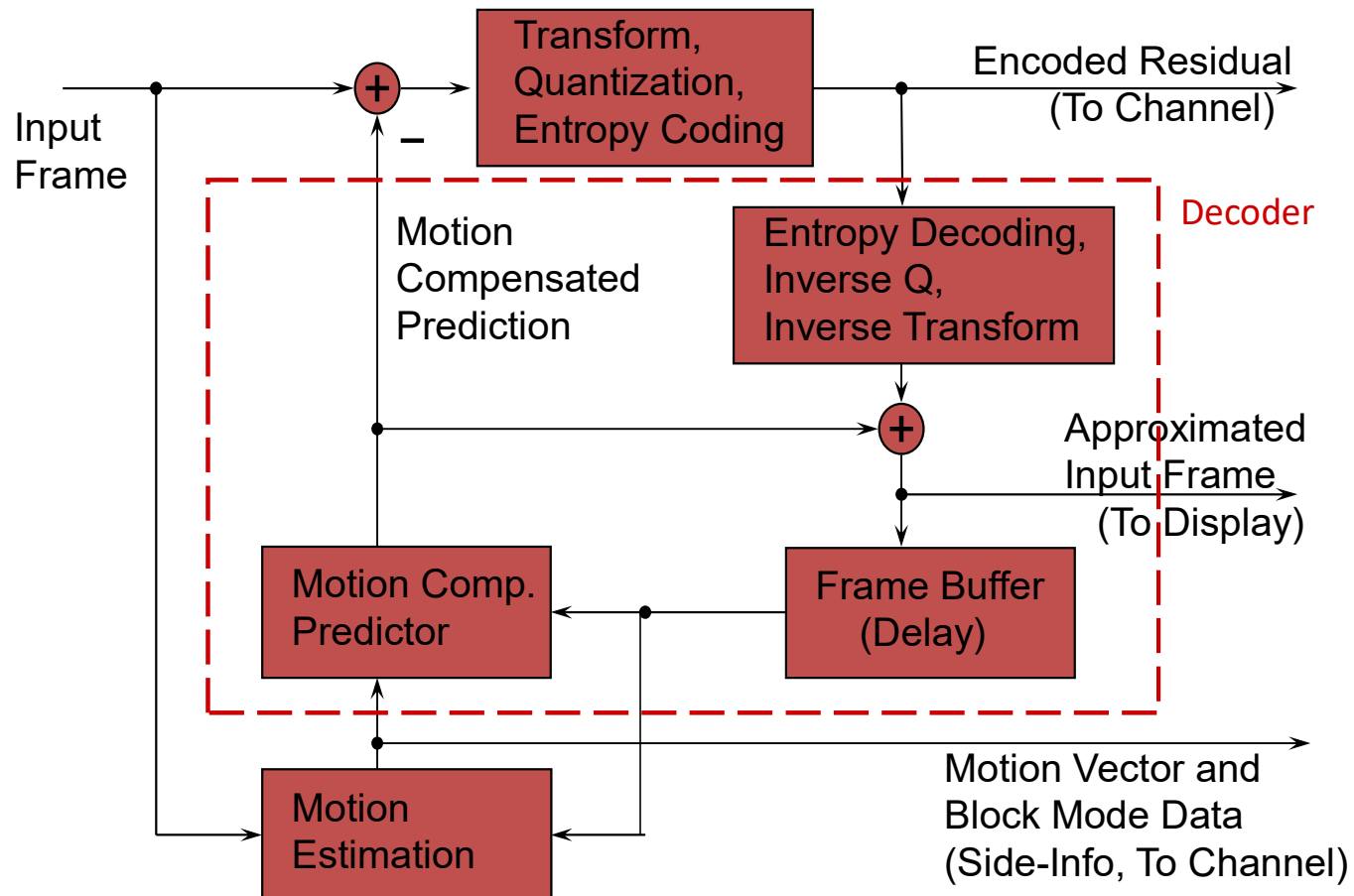
Shearing

Application of Geometric Transformation

Motion-compensated Video Coding



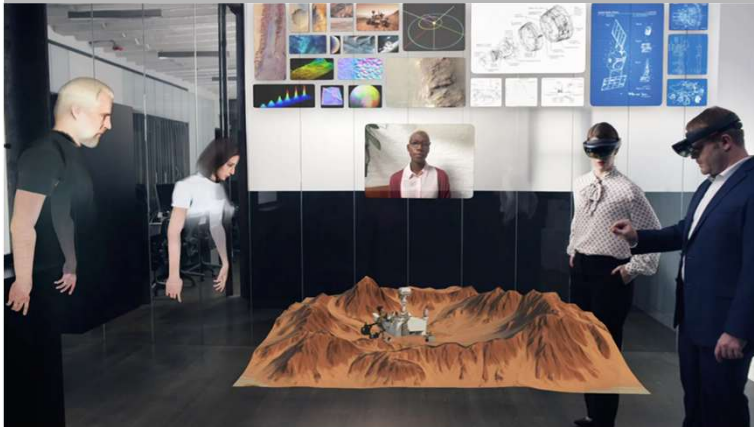
Typical Motion-Compensation Codec (MPEG2, H.264, SVC)



Application of Geometric Transformation

3D Dynamic Mesh Compression

3D Virtual Meeting



Virtual Reality

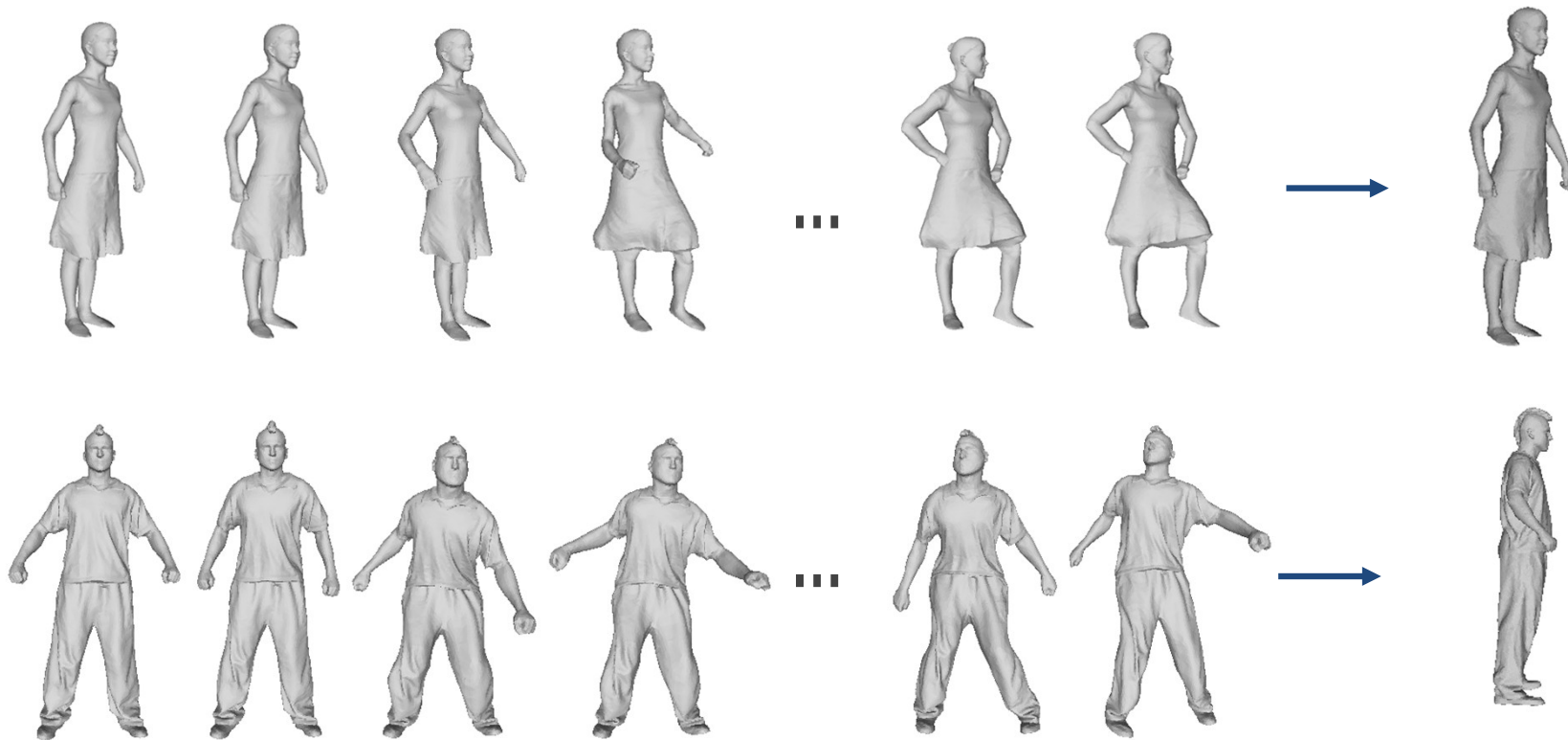


3D Gaming



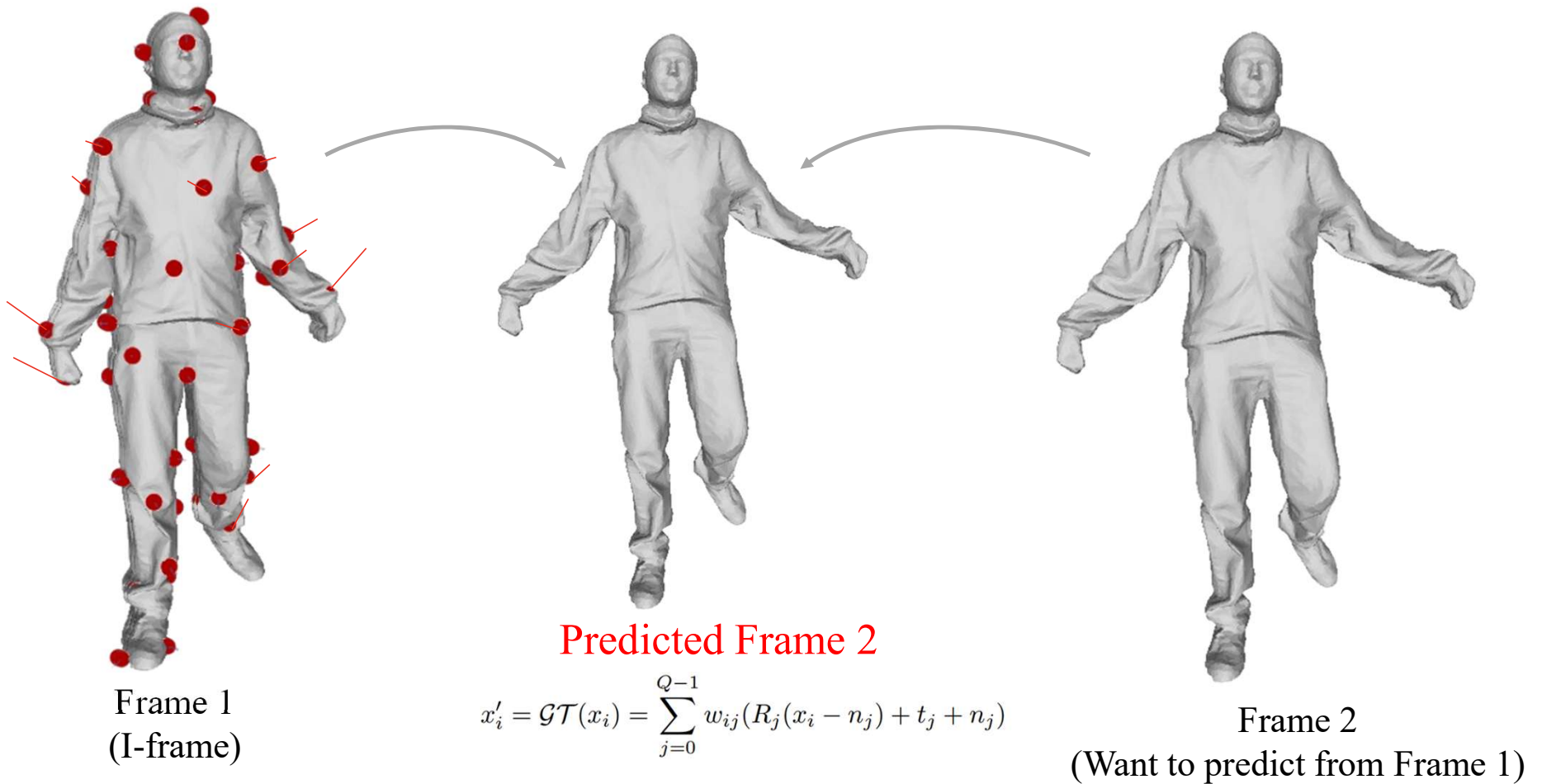
3D Dynamic Mesh Examples

(Skipped some frames in between to show motions)



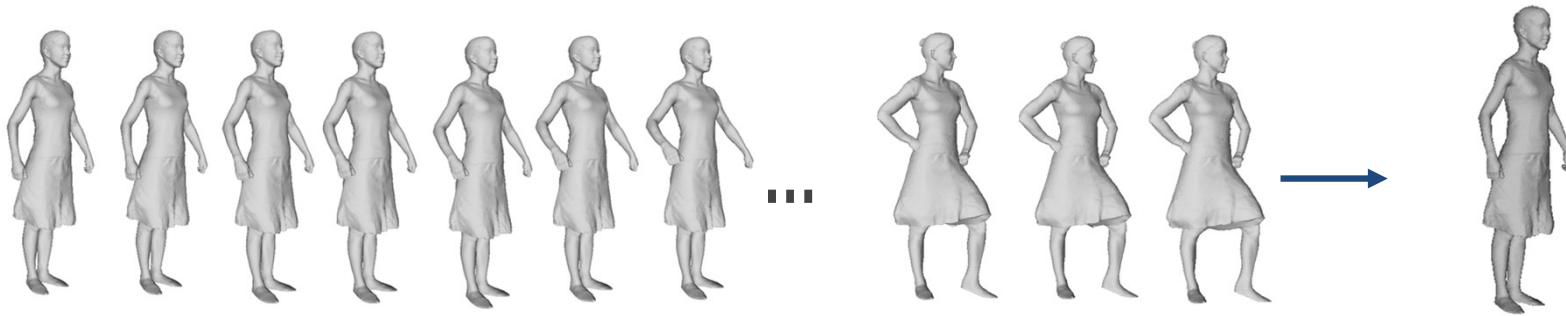
Courtesy of Huong Hoang

Keynode-Driven Dynamic Mesh Compression

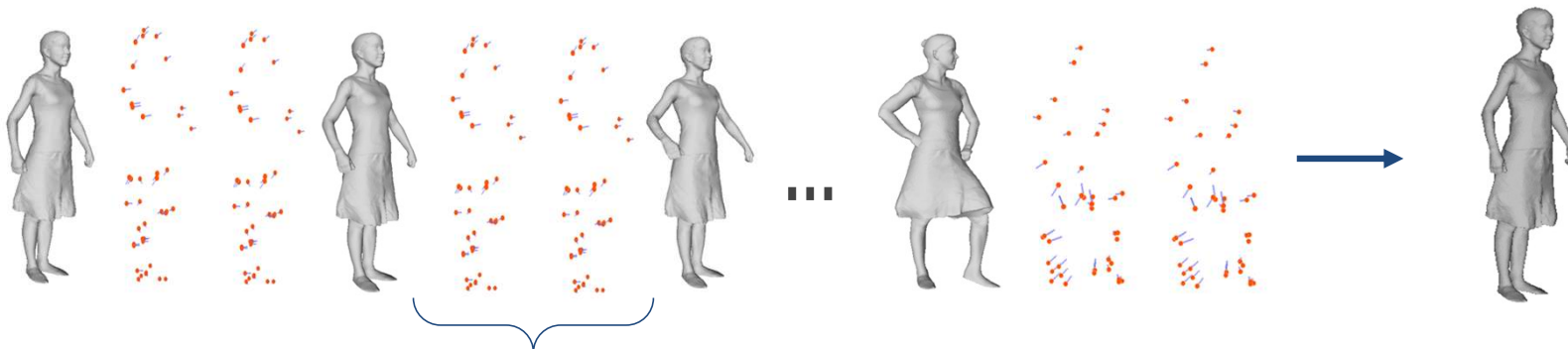


Keynode-Driven Dynamic Mesh Compression

Original Dynamic Mesh:

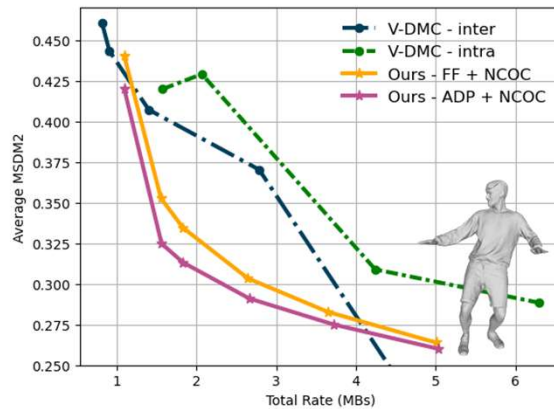


Keynode-Driven Dynamic Mesh Compression:

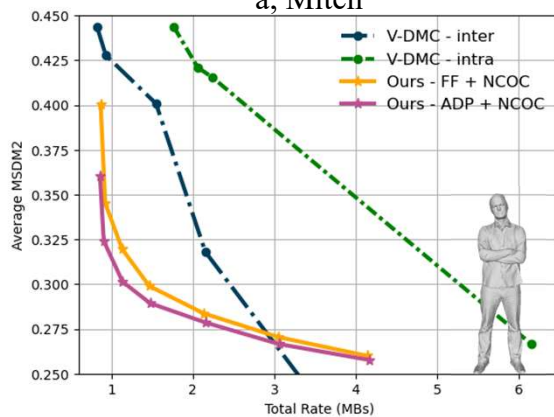


Much smaller in size than the original frames

Keynode-Driven Dynamic Mesh Compression



a, Mitch



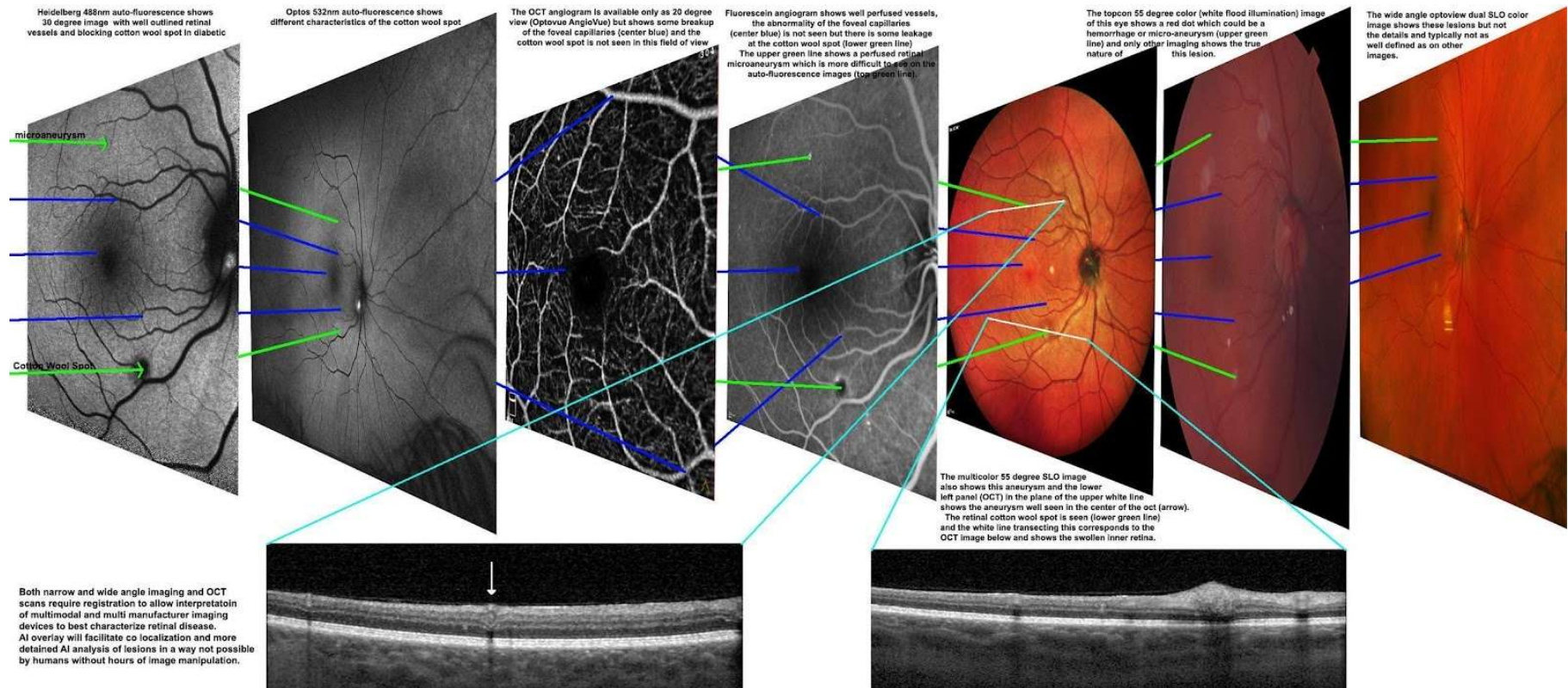
b, Thomas



Courtesy of Huong Hoang

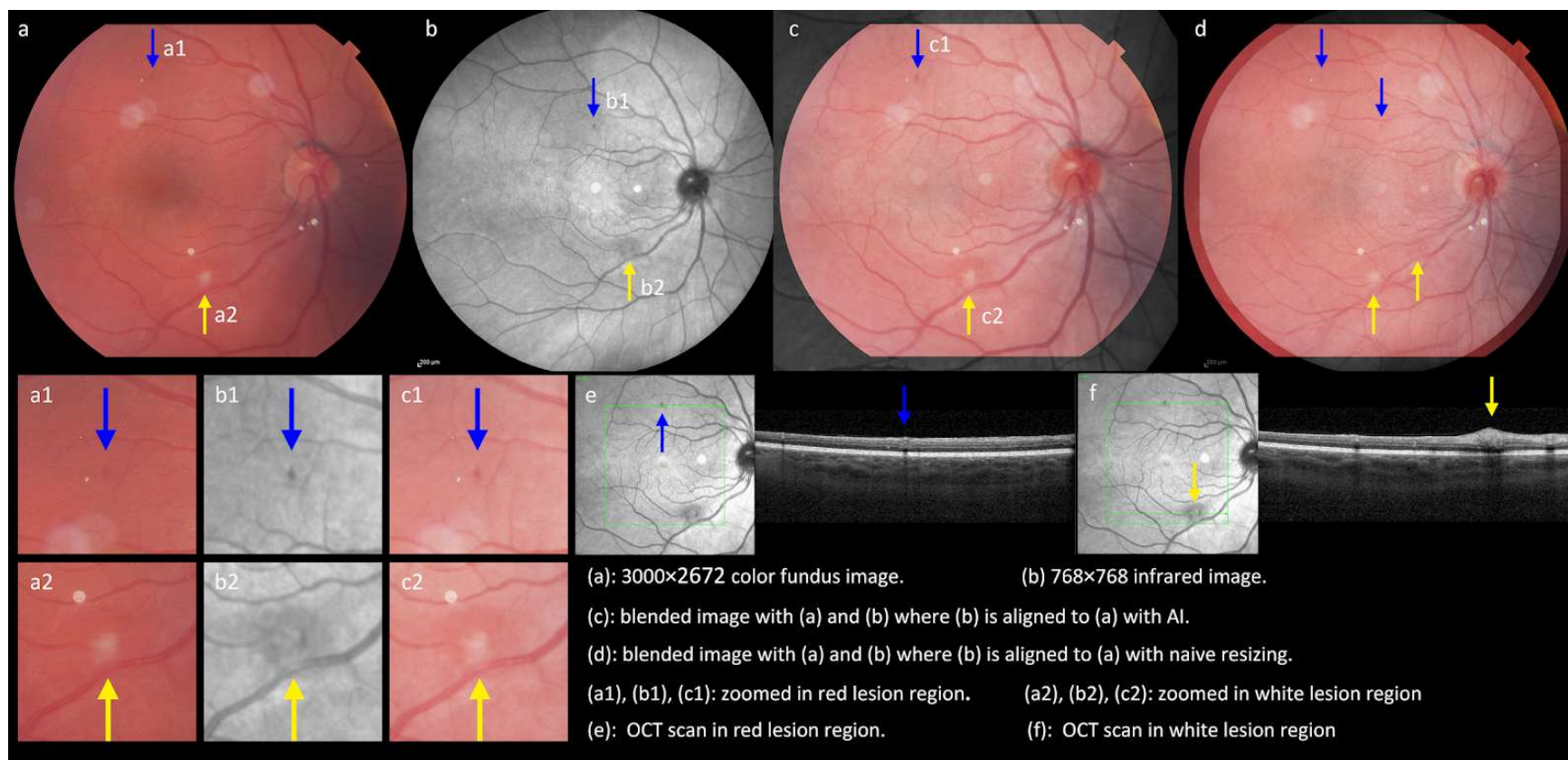
Application of Geometric Transformation

Multimodal Retina Image Registration



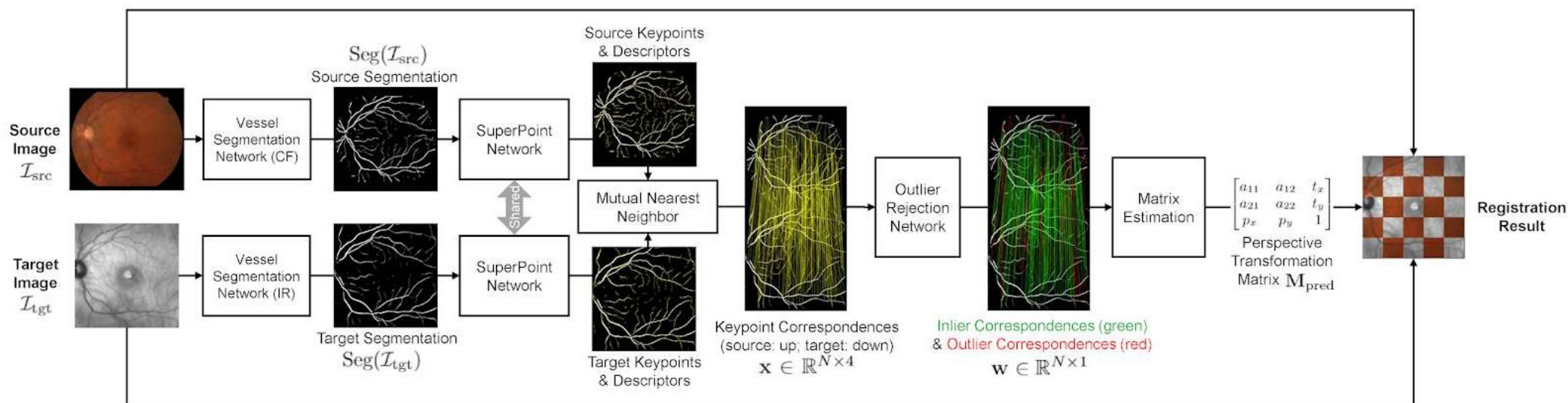
Overlay of multimodal imaging onto an SLO image. This is done manually by an ophthalmologist, doing so with OCT and the multimodal imaging would take 3-5 minutes per lesion, or over 30 minutes per eye for a diabetic with ten lesions in each eye.

Application of Geometric Transformation



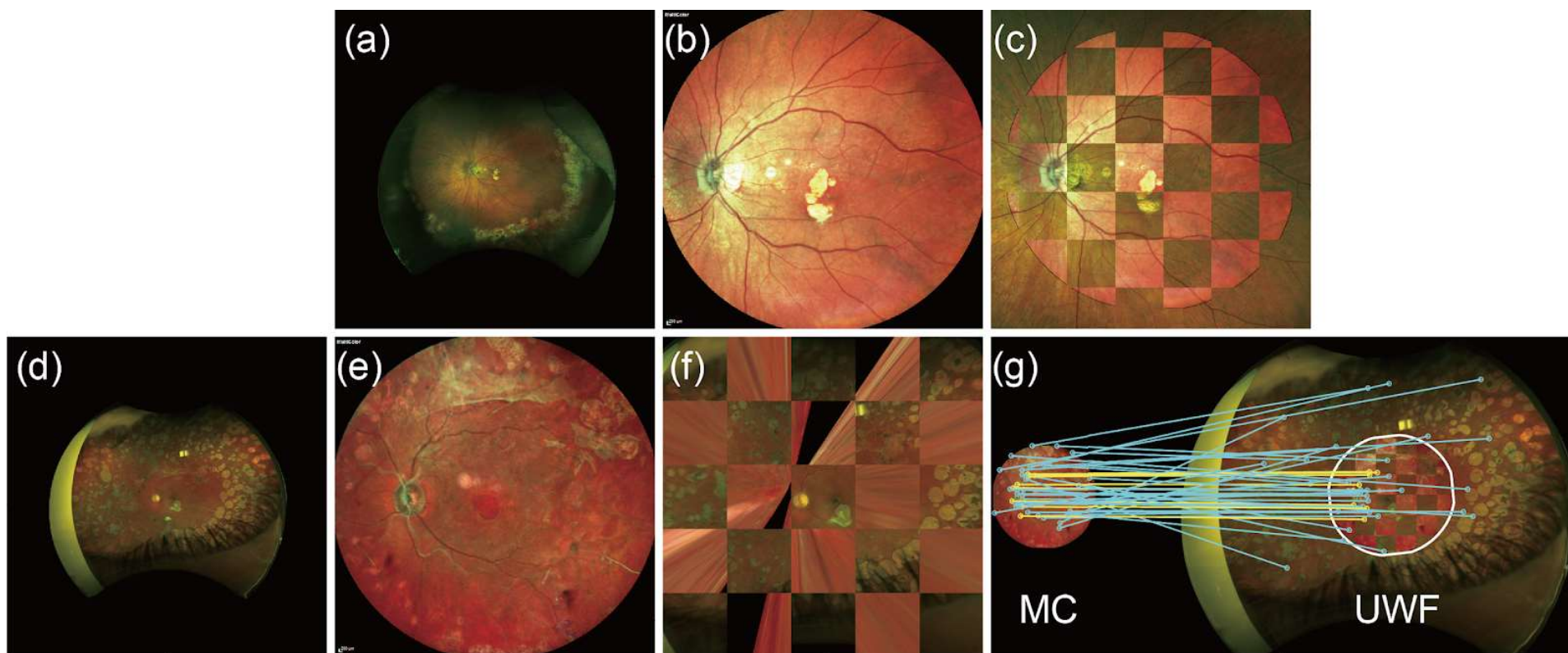
Alignment of color fundus (a), infrared image (b), and OCT scans (e), (f). The images reveal red lesion (blue arrow) and white lesion (yellow arrow) in a diabetic patient.

Application of Geometric Transformation



Learning-based multimodal retinal image registration pipeline

Application of Geometric Transformation



Results of aligning MC to UWF. (a) UWF; (b) MC; (c) zoomed-in overlay of successfully registered images from (a) and (b); (d) UWF; (e) MC; (f) failure registration results of (d) and (e); (g) feature matching results of (d) and (e) where yellow and blue lines indicate correct and wrong matches, and white circle indicates the ground truth alignment of MC to UWF.

Deformation-based Dynamic Mesh Compression and Applications in XR

Team Members:

Kunyao Chen

Bang Du

Huong Hoang

Fei(Zack) Yin

Baichuan(Peter) Wu

Girish Krishnan



Human Digitization From Animation to VR/AR/Metaverse



Movie



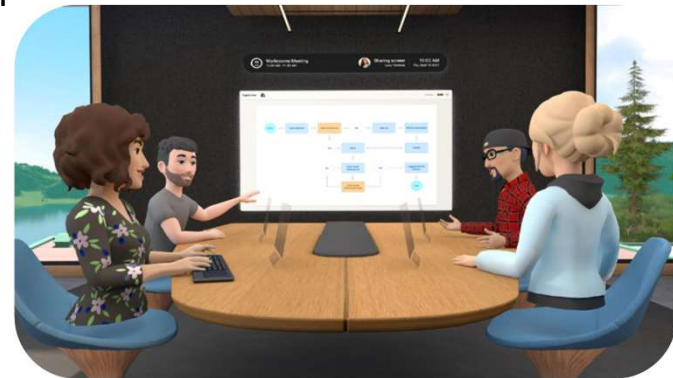
VTube
r



Virtual Event

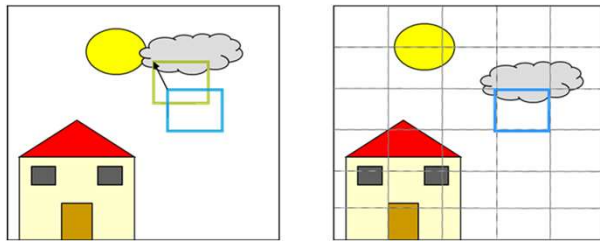


Video Game

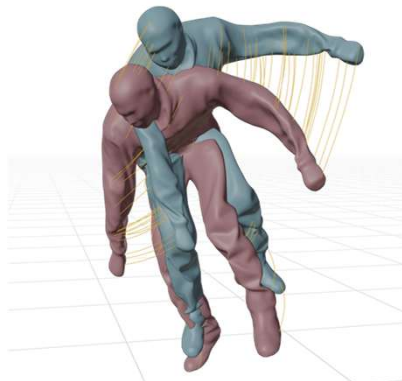


Meta: Workrooms

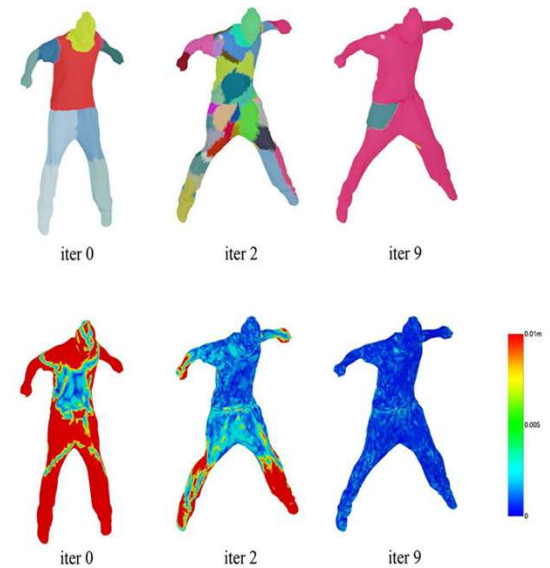
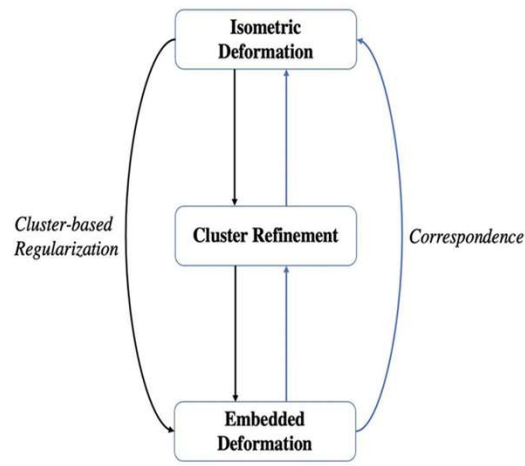
Robust Region-Adaptive Deformation



2D Video: Motion Vectors



3D Human Mesh: Deformation vertices



Robust Region-adaptive Deformation

K. Chen, F. Yin, B. Du, B. Wu and T. Nguyen, "Efficient Registration for Human Surface via Isometric Regularization on Embedded Deformation," IEEE Trans. on Visualization and Computer Graphics, 2022.

Related Work: Embedded Deformation

In dense deformation^[13]:

$\omega(p)$ is parameterized with per-vertex affine matrix $\{A_i | A_i \in \mathbb{R}^{3 \times 3}\}$ and translation $\{t_i | t_i \in \mathbb{R}^3\}$
Each vertex x_i is mapped to new position x_i^* in the K iteration:

$$x_i^K = A_i x_i + t_i$$

In embedded deformation^[14]:

A down sampled structure is adopted with nodes $\{n_j | n_j \in \mathbb{R}^3\}$

$\omega(p)$ is parameterized with *per-node* affine matrix $\{A_j | A_j \in \mathbb{R}^{3 \times 3}\}$ and translation $\{t_j | t_j \in \mathbb{R}^3\}$

Each vertex x_i is mapped to new position x_i^K in the K iteration:

$$x_i^K = \sum_j \bar{w}_{ij} \left(A_j (x_i - n_j) + t_j + n_j \right)$$

[13] O. Sorkine, M. Alexa, As-rigid-as-possible surface modeling, in: Symposium on Geometry Processing, 2007: pp. 109–116.

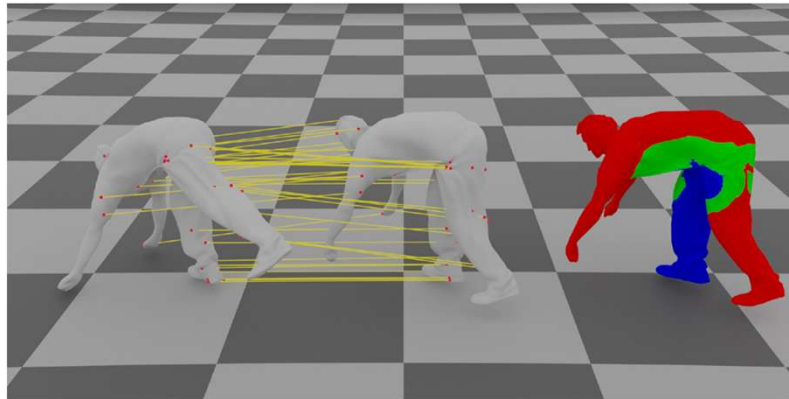
[14] R.W. Sumner, J. Schmid, M. Pauly, Embedded deformation for shape manipulation, in: ACM Siggraph 2007 Papers, 2007: pp. 80-es.

Problem of Embedded Deformation

The regularization of embedded deformation is defined on nodes:

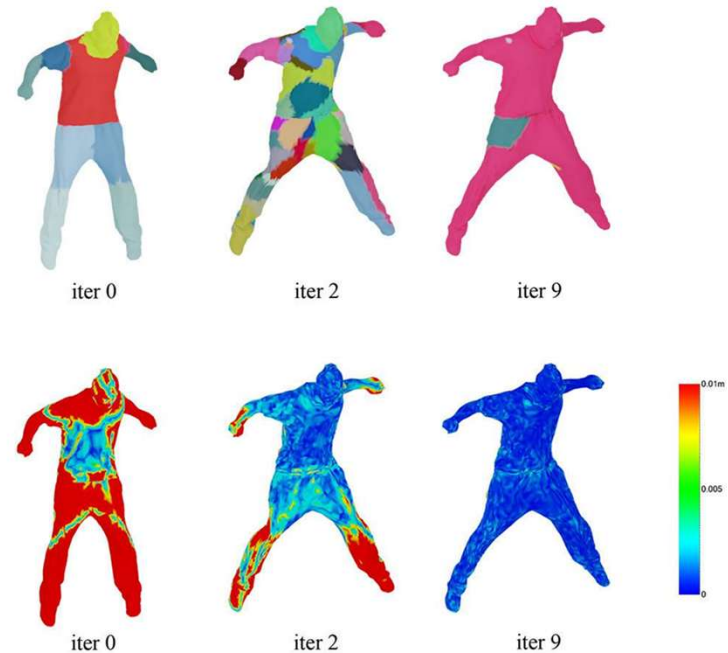
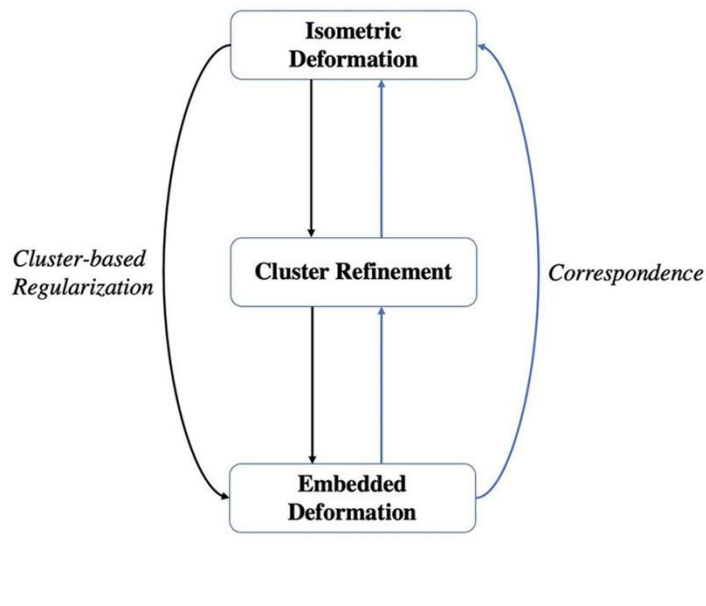
$$\mathcal{L}_{reg} = \sum_k \sum_j w_{kj} \|A_j(n_k - n_j) + t_j + n_j - n_k - t_k\|^2$$

1. It may not reflect the surface rigidity accurately
2. Changing the nodes will affect regularization



In human cases, large-motion regions should deform more rigidly (red), full-detail regions should deform less rigidly (green & blue)

Proposed Method



Contributions:

1. Decouple regularization from underlying deformation model
2. Achieve region-adaptive regularization
3. Require little overhead

Isometric Deformation

In isometric deformation(first step): each vertex is assigned with a cluster and deform rigidly with its cluster $c(i)$

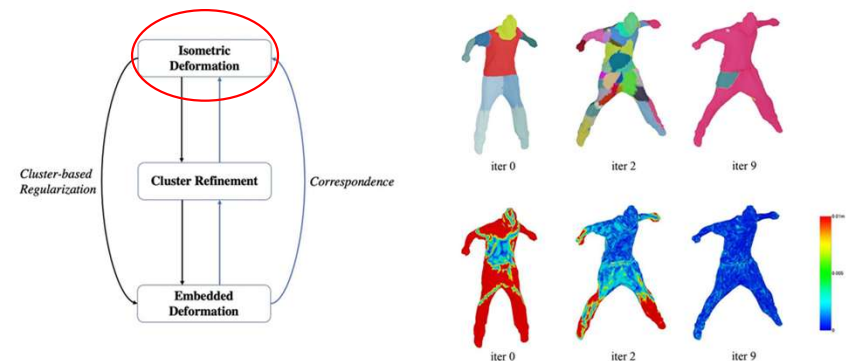
$$\dot{x}_i^K = A_{c(i)}^K x_i^{K-1} + t_{c(i)}^K$$

Cluster-based affine matrix $\{A_{c(i)} | A_{c(i)} \in \mathbb{R}^{3 \times 3}\}$ and translation $\{t_{c(i)} | t_{c(i)} \in \mathbb{R}^3\}$ are estimated **with inter-cluster regularization:**

$$\mathcal{L}_{reg(c)} = \sum_i \sum_j \|A_{c(i)}^K x_i^{K-1} + t_{c(i)}^K - A_{c(j)}^K x_i^{K-1} - t_{c(j)}^K\|^2$$

Cluster $c(i)$ are refined give $\mathcal{L}_{reg(c)}$:

Clusters are combined if $\mathcal{L}_{reg(c)}$ is small



Embedded Deformation with Cluster-based Regularization

In embedded deformation(second step): each vertex deforms non-rigidly

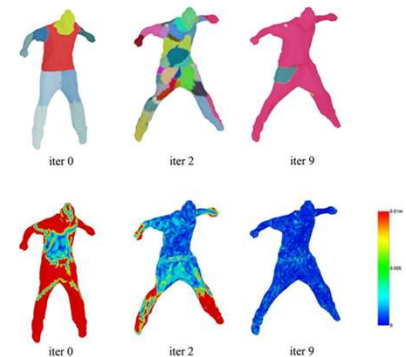
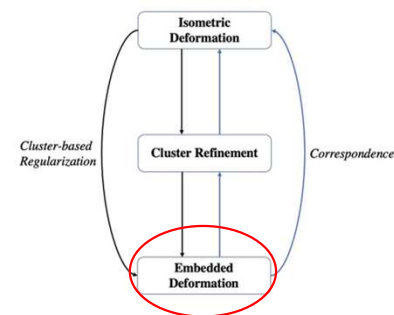
$$x_i^K = \sum_j \bar{w}_{ij} \left(A_j(x_i - n_j) + t_j + n_j \right)$$

Regularized with $A_{c(i)}$ and $t_{c(i)}$

$$\mathcal{L}_{reg(n)} = \sum_i \|A_{c(i)}^K x_i^{K-1} + t_{c(i)}^K - x_i^K\|^2 = \sum_i \|\dot{x}_i^K - x_i^K\|^2$$

Cluster $c(i)$ are refined given intra-cluster $\mathcal{L}_{reg(n)}$:

Clusters are separated if $\mathcal{L}_{reg(n)}$ is large



Experiments

- MIT Human Dataset^[15]
 - Capture @ 25Hz
- Three test cases:
 - General: 500 samples, 1 frame apart (25Hz)
 - Incomplete: 500 samples(with randomly generated holes), 1 frame apart(25Hz)
 - Large-displacement: 300 samples, 5 frame apart(5Hz)
- Initialization:
 - General: one cluster, random value
 - Incomplete: one cluster, random value
 - Large-displacement: ten clusters, estimated parameters(also apply to other methods)
- Metric:

$$f(A, B) = \sqrt{\frac{1}{N_v} \sum_{x \in \mathcal{V}_A} (\min_{p \in \mathcal{S}_B} \|x - p\|^2)}$$

$$RMSE = f(A, B) + f(B, A)$$



One frame apart v.s. five frame apart

[15] D. Vlasic, I. Baran, W. Matusik, J. Popović, Articulated mesh animation from multi-view silhouettes, in: ACM SIGGRAPH 2008 Papers, 2008: pp. 1–9.

Numerical Results

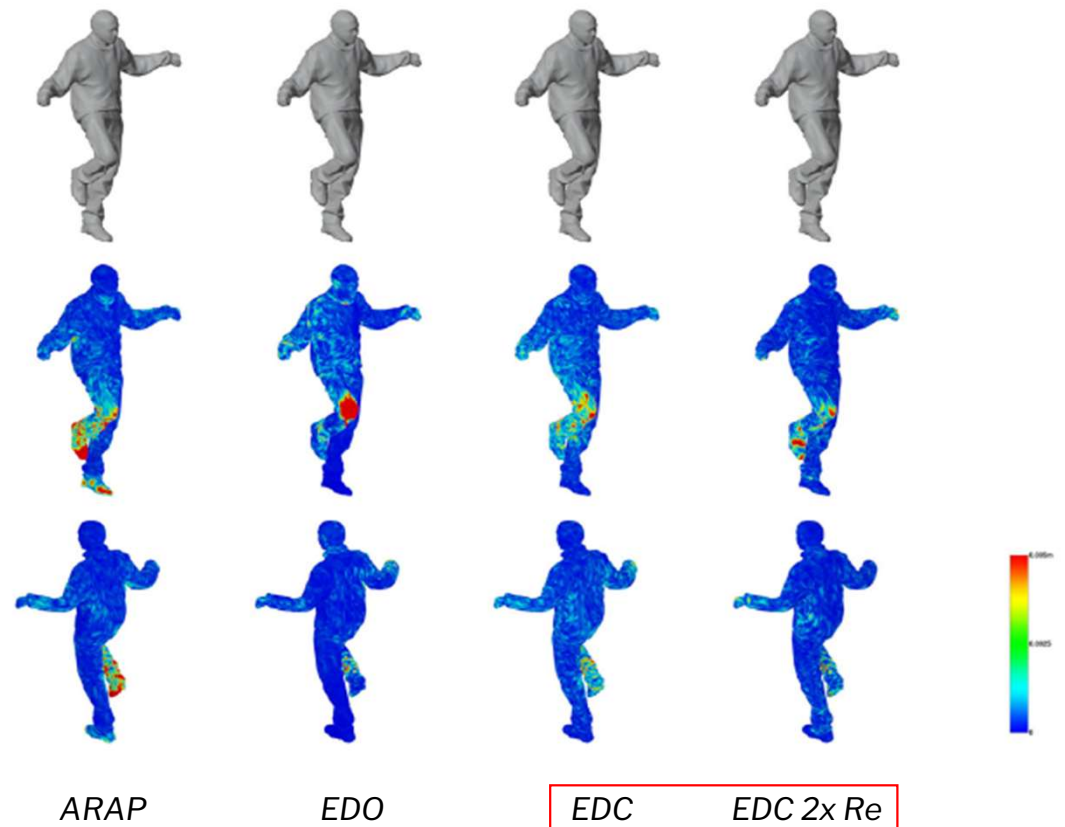
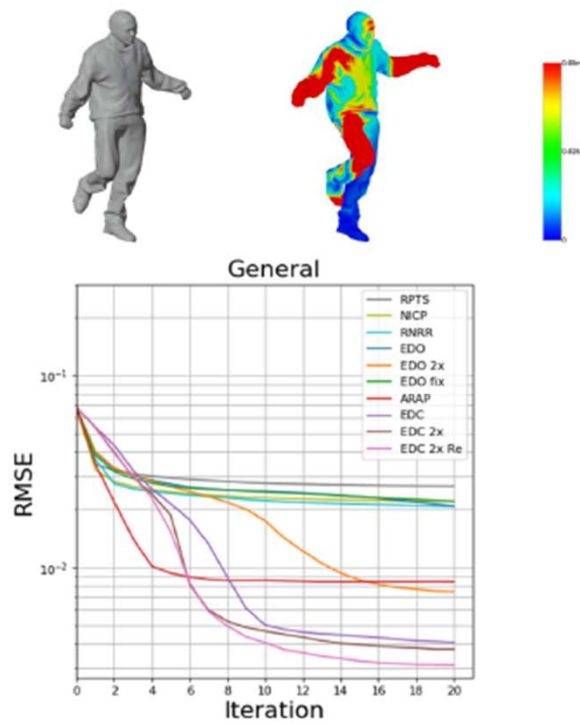
	General	Incomplete Mesh	Inaccurate Corr
RPTS	0.008805	0.008989	/
NICP	0.008311	0.008449	/
RNRR	0.007295	0.007602	/
EDO	0.003839	0.004500	0.006014
EDO 2x	0.004367	0.005155	0.006783
EDO fix	0.004168	0.004909	0.006327
ARAP	0.003835	0.004980	0.005809
EDC	0.003386	0.003541	0.004408
EDC 2x	0.003042	0.003707	0.003624
EDC 2x Re	0.002775	0.003325	0.003086

Proposed methods (EDCs) outperform other methods

	Time (ms)	# Parameters
EDO	23.408	6000
ARAP	153.966	140028
RNRR	357.440	/
EDC Cluster	4.874	<600
EDC Nonrigid	20.051	6000
EDC Combined	24.925	6000

Proposed methods are very efficient

Qualitative Results



Multirate DSP

- Upsampling and downsampling
- Interpolation and decimation

Next Lecture

- Intensity transformations
- Reading
 - Chapter 3: Intensity Transformations and Spatial Filtering
 - Sections 3.1, 3.2, and 3.3